



я сделал такие CSV файлы с url и сниппетами страничек. я хочу подключить какую то локальную модельку с hugging face для того, чтобы она анализировала, насколько они соответствуют запросу query про БД db в той же строчке. по сути - у меня задача классификации, где мне нужно определить страничку к одной из 3 категорий - полезно, спам, неизвестно. как ты думаешь, для этого лучше подойдёт nlp моделька или небольшая llm? Для лучшего понимания ситуации, на чём я буду это запускать - у меня ноутбук с 16 Гб оперативной памяти и rtx 3050ti

Для твоей задачи я бы **брал не маленькую LLM, а обычную NLP-модель ранжирования/классификации** — лучше всего bi-encoder или, ещё лучше, cross-encoder/reranker. В твоих CSV каждая строка уже выглядит как структурированная пара “query + db + url + snippet”, а сами сниппеты местами шумные, обрезанные и с ложными совпадениями вроде страниц про “Big Data at Berkeley”, общих справок, маркетинговых страниц и нерелевантных результатов, поэтому задача ближе к relevance classification, чем к генерации текста.^{[1] [2]}

Что выбрать

Для локального запуска на ноутбуке с 16 ГБ RAM и RTX 3050 Ti практичнее всего будет небольшой encoder-модельный стек, потому что тебе нужно много раз прогонять короткие тексты и получать стабильный класс, а не генерировать объяснения. По самим примерам из файлов видно, что основная трудность — не “понять длинный документ”, а отличить полезный результат от спама/ложного совпадения по query, db, url и snippet в одной строке.^{[2] [1]}

Лучший базовый вариант:

- bi-encoder для быстрого первичного скоринга;
- cross-encoder/reranker для финального решения по топ-k;
- отдельное правило или порог для класса “неизвестно”.

Почему не LLM

Маленькая LLM тоже сможет это делать, но для такой задачи она обычно медленнее, менее предсказуема и хуже масштабируется на большом CSV, чем компактная модель ранжирования. Особенно это заметно, когда данные шумные: в твоих файлах много строк, где есть поверхностное совпадение по словам, но не по смыслу, и тут хорошо работает именно pairwise relevance-модель, а не генеративный ответ модели. ^[1] ^[2]

LLM я бы подключал только в двух случаях:

- если тебе нужно ещё и объяснение “почему это спам/полезно”;
- если потом захочешь few-shot без разметки и быстро проверить гипотезу на маленькой выборке.

Конкретные модели

Если хочется прямо сейчас собрать рабочий пайплайн на Hugging Face, я бы смотрел в такую сторону:

- BAAI/bge-reranker-base или аналогичный reranker;
- cross-encoder/ms-marco-MiniLM-L-6-v2 как лёгкий и быстрый baseline;
- sentence-transformers модели типа bge-small / multilingual-e5-small для первичного retrieval/scoring.

Если у тебя данные русско-английские вперемешку, multilingual-модель будет безопаснее, потому что в CSV заметны и английские, и русские, и местами смешанные/ломанные сниппеты. ^[2] ^[1]

Практичная схема

Я бы делал так:

1. Собери текст пары:
query [SEP] db [SEP] title/url [SEP] snippet
2. Разметь хотя бы 500–1500 строк вручную в 3 класса:
 - полезно,
 - спам,
 - неизвестно.
3. Обучи сначала простой baseline:
 - TF-IDF + Logistic Regression,
 - потом MiniLM/cross-encoder.
4. Для “неизвестно” не заставляй модель угадывать жёстко, а вводи порог уверенности, потому что в твоих данных реально много пограничных случаев и мусорных сниппетов. ^[1] ^[2]

Мой совет

Если нужен лучший баланс качества, скорости и простоты, бери **маленький cross-encoder** как основную модель. Если нужен ещё более быстрый продюсовый вариант на больших объёмах, делай двухэтапную схему: **bi-encoder** для отбора кандидатов и **cross-encoder** для финального класса.

Для твоего железа это будет заметно удобнее, чем гонять маленькую LLM локально на каждой строке. Хочешь — я могу сразу предложить готовый пайплайн на Python с **transformers** для твоих CSV: загрузка, формирование признаков, обучение и инференс по классам `useful/spam/unknown`.



а если у меня нет времени на разметку данных и обучение модели?

Да — тогда я бы вообще **не обучал модель**, а пошёл в **zero-shot** или в готовый **reranker**. Для твоей задачи это самый реалистичный путь: Hugging Face прямо описывает **zero-shot classification** как способ классифицировать тексты по новым меткам без собственных размеченных примеров, а **cross-encoder** можно использовать как готовую модель для оценки релевантности пары `query-document`.^[3]
^[4] ^[5]

Лучший путь без обучения

Если нужен самый практичный вариант без разметки, я бы начал с **готового cross-encoder reranker**: он принимает `query` и текст документа вместе и возвращает `score` релевантности, что хорошо совпадает с твоей постановкой “насколько страничка соответствует запросу”. Для русских данных есть готовые модели вроде `DiTy/cross-encoder-russian-msmarco`, а для общего **baseline** есть `cross-encoder/ms-marco-MiniLM-L-6-v2`; Hugging Face и Sentence Transformers показывают именно такой способ использования.^[6] ^[7]

Схема тогда простая:

- подаёшь `query + db` как запрос;
- подаёшь `title/url/snippet` как документ;
- получаешь `score`;
- по порогам переводишь его в `полезно / неизвестно / спам`.

Такой подход не требует обучения, а только подбора порогов на 30–50 строках вручную “на глаз”.^[4]
^[6]

Когда брать zero-shot

Zero-shot classification хорош, если тебе хочется сразу получить один из трёх классов по меткам `useful`, `spam`, `unknown` без собственного датасета. Hugging Face описывает это как классификацию по заранее заданным **candidate labels** без примеров обучения, а в документации рекомендуют модели вроде `facebook/bart-large-mnli` и **multilingual zero-shot** модели.^[8] ^[3]

Но для твоего кейса zero-shot обычно хуже, чем reranker, потому что у тебя задача не “о чём текст”, а “релевантен ли этот документ данному запросу”. То есть zero-shot — это хороший быстрый старт, а cross-encoder — обычно более точный путь для search relevance. ^[3] ^[4]

Что я бы выбрал тебе

На ноутбуке с RTX 3050 Ti я бы начал с небольшого cross-encoder, а не с LLM и не с тяжёлым zero-shot на большой NLI-модели. Cross-encoders дают высокую точность именно в relevance ranking, но они вычислительно дороже, поэтому лучше прогонять ими строки батчами и не ждать от них скорости как у TF-IDF. ^[7] ^[4]

Мой практический приоритет был бы таким:

1. DiTy/cross-encoder-russian-msmarco, если запросы и сниппеты часто на русском. ^[6]
2. cross-encoder/ms-marco-MiniLM-L-6-v2, если много английского контента или нужен лёгкий baseline. ^[7]
3. Zero-shot только если хочешь сразу 3 класса без ручной логики порогов. ^[8] ^[3]

Как разложить на 3 класса

Без обучения я бы не просил модель сразу идеально различать спам и неизвестно, а сделал это через score:

- высокий score → полезно;
- средний score → неизвестно;
- низкий score → спам.

Это обычно устойчивее, чем пытаться навязать zero-shot модели тонкую грань между “нерелевантно” и “совсем спам”, особенно на шумных сниппетах и агрегаторах. ^[4]

Примерно так можно мыслить:

- score > 0.7 → полезно
- 0.35–0.7 → неизвестно
- < 0.35 → спам

Точные пороги всё равно лучше быстро проверить на маленькой выборке вручную, потому что они зависят от конкретной модели и качества твоих сниппетов. ^[6] ^[7]

Мой итог

Если совсем нет времени на разметку, бери **готовый cross-encoder и пороги**, это самый здравый вариант. Zero-shot тоже рабочий, но скорее как быстрый черновой baseline, тогда как reranker лучше соответствует самой природе твоей задачи — оценке релевантности query-document пары. ^[3] ^[4]

Могу сразу дать тебе готовый Python-скрипт для CSV, который:

- читает query, db, url, snippet,
- считает score через локальную HF-модель,

- раскладывает строки по полезно / неизвестно / спам,
- сохраняет новый CSV.

✱✱

давай тогда действовать по шагам. начнём с выбора и установки конкретной модели и библиотек

Да, начнём с самого практичного варианта: ставим `cross-encoder` через `sentence-transformers` и берём одну конкретную модель как старт. Для твоего кейса я бы выбрал `DiTy/cross-encoder-russian-msmarco` как первый кандидат, потому что она уже предназначена для `pairwise relevance scoring` и на странице модели есть прямой пример использования через `CrossEncoder(..., device='cuda')`; как запасной `baseline` можно держать `cross-encoder/ms-marco-MiniLM-L6-v2`. [\[33\]](#) [\[34\]](#) [\[35\]](#)

Что ставим

Минимальный стек такой:

- Python 3.10+;
- PyTorch с CUDA, если хочешь использовать RTX 3050 Ti;
- `sentence-transformers`;
- `transformers`, `accelerate`, `pandas`.

Hugging Face рекомендует ставить `transformers` в отдельном виртуальном окружении и отдельно проверять доступность NVIDIA GPU через `nvidia-smi`, а для `cross-encoder`-моделей достаточно `sentence-transformers`, который уже даёт удобный API `CrossEncoder`. [\[36\]](#) [\[37\]](#) [\[38\]](#)

Какую модель выбрать первой

Я бы **первой** ставил `DiTy/cross-encoder-russian-msmarco`, потому что у тебя запросы и сниппеты могут быть русско-английскими вперемешку, а модель явно ориентирована на русский `retrieval/re-ranking` сценарий. Если увидим, что англоязычные результаты она оценивает слабо, вторым шагом добавим `cross-encoder/ms-marco-MiniLM-L6-v2` и сравним их на небольшой выборке. [\[34\]](#) [\[33\]](#)

Команды установки

Если у тебя Windows и обычный `venv`, самый безопасный старт такой:

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install -U pip
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/wheels
pip install -U sentence-transformers transformers accelerate pandas
```

`sentence-transformers` ставится отдельно одной командой, а `transformers` официально работает с PyTorch и `virtual environment`; для GPU-ускорения Hugging Face отдельно советует убедиться, что CUDA и NVIDIA GPU определяются системой. [\[37\]](#) [\[38\]](#) [\[36\]](#)

Если у тебя Linux, то активация окружения будет такой:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -U pip
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/wheels
pip install -U sentence-transformers transformers accelerate pandas
```

Проверка после установки

После установки проверь сначала GPU:

```
nvidia-smi
```

Потом проверь, что библиотека импортируется и модель грузится:

```
from sentence_transformers import CrossEncoder
model = CrossEncoder("DiTy/cross-encoder-russian-msmarco", max_length=512, device="cpu")
print("ok")
```

Страница модели `DiTy/cross-encoder-russian-msmarco` показывает именно такой способ запуска, а документация Sentence Transformers подтверждает, что `CrossEncoder` принимает пары (`query`, `document`) и возвращает `scores`. [\[35\]](#) [\[33\]](#)

Что делаем дальше

После установки следующий шаг будет очень маленький и правильный: загрузить **5–10 строк** из твоего CSV и проверить, как модель оценивает пары `query+db` против `snippet/url`. Это лучше, чем сразу писать весь пайплайн, потому что мы быстро поймём, подходит ли первая модель по качеству и скорости. [\[39\]](#) [\[40\]](#) [\[33\]](#)

Скажи, у тебя сейчас Windows или Linux, и я дам тебе **точные команды под твою ОС**, а потом сразу перейдём к тестовому скрипту загрузки модели.



окей, я вроде как всё установил и запустил то, что ты скинул выше, пока все работает

Отлично, тогда следующий шаг — **не трогать пока весь CSV**, а проверить модель на нескольких строках вручную. По содержимому файлов видно, что у тебя строки фактически представляют связку `db + query + url + snippet`, причём данные местами грязные, слепленные и с шумом, поэтому сначала важно убедиться, что модель вообще адекватно ранжирует такие пары. [\[50\]](#) [\[51\]](#)

Что делаем сейчас

Наша цель на этом шаге:

- загрузить модель;
- подать 5–10 тестовых пар;
- посмотреть SCORE;
- понять, похожи ли оценки на здравый смысл.

Это важно, потому что в файлах есть и явно полезные страницы, и мусор вроде агрегаторов, курсов, YouTube, переводов, случайных общих статей и ложных совпадений. [\[51\]](#) [\[50\]](#)

Тестовый скрипт

Скопируй и запусти такой минимальный скрипт:

```
from sentence_transformers import CrossEncoder

model = CrossEncoder(
    "DiTy/cross-encoder-russian-msmarco",
    max_length=512,
    device="cuda"
)

pairs = [
    [
        "MySQL Big Data",
        "https://dba.stackexchange.com/questions/114539/mysql-big-data-solutions It c
    ],
    [
        "MySQL Big Data",
        "https://www.coursera.org/learn/analytics-mysql Offered by Duke University.
    ],
    [
        "Berkeley DB Big Data",
        "https://debrabernier.com/navigating-the-depths-oracle-berkeley-db-and-big-
    ],
    [
        "Berkeley DB Big Data",
        "https://bd.studentorg.berkeley.edu Big Data at Berkeley is a UC Berkeley s
    ],
    [
        "PostgreSQL Big Data",
        "https://www.enterprisedb.com/blog/powering-big-data-processing-postgres-ap
    ]
]

scores = model.predict(pairs)

for pair, score in zip(pairs, scores):
    print("=" * 80)
    print("QUERY:", pair[0])
```

```
print("TEXT :", pair[^4_1][:200], "...")
print("SCORE:", float(score))
```

Этот тест хорошо отражает типы строк, которые реально есть у тебя в CSV: полезные технические материалы про MySQL Big Data и PostgreSQL Big Data, а также ложные совпадения вроде Big Data at Berkeley, которое по словам совпадает, но по смыслу не про Berkeley DB.^[50] ^[51]

Как интерпретировать результат

Ты пока не смотри на абсолютную “правильность” числа, смотри на **порядок**:

- техническая статья по нужной БД должна иметь score выше;
- курс, агрегатор или случайное совпадение должны быть ниже;
- пограничные случаи будут рядом посередине.

Например, страница про Oracle Berkeley DB and big data management должна быть выше, чем страница студенческой организации Big Data at Berkeley, потому что в файлах у тебя такие ложные совпадения действительно встречаются.^[50]

Что мне прислать дальше

Пришли:

- 5–10 полученных score;
- или просто вывод консоли целиком;
- и коротко, какие из них тебе кажутся “странными”.

После этого мы сделаем следующий шаг: аккуратно подключим чтение твоего CSV и прогоним модель уже по реальным строкам батчами.

✱✱

Так, вот консольный вывод:

QUERY: MySQL Big Data

TEXT : <https://dba.stackexchange.com/questions/114539/mysql-big-data-solutions> It depends on your table structures, data layout and usage pattern.

However I wouldn't consider 12 gb ... I work with MySQL serv ...

SCORE: 0.8978384733200073

QUERY: MySQL Big Data

TEXT : [https://www.coursera.org/learn/analytics-mys](https://www.coursera.org/learn/analytics-mysql)

q| Offered by Duke University. This course is an introduction to how to use relational databases in business analysis. ...

SCORE: 0.0018864368321374059

QUERY: Berkeley DB Big Data

TEXT : [https://debrabernier.com/navigating-the-depths](https://debrabernier.com/navigating-the-depths-oracle-berkeley-db-and-big-data-management)

-oracle-berkeley-db-and-big-data-management Oracle Berkeley DB ... robust solution for handling large volumes of data in big data management. ...

SCORE: 0.8268426656723022

QUERY: Berkeley DB Big Data

TEXT : <https://bd.studentorg.berkeley.edu> Big Data at Berkeley is a UC Berkeley student organization dedicated to promoting data science... ..

SCORE: 0.9722841382026672

QUERY: PostgreSQL Big Data

TEXT : <https://www.enterprisedb.com/blog/powering-big-data-processing-postgres-apache-spark> FDWs act as pipelines connecting Postgres with external database solutions including Hadoop. ...

SCORE: 0.48734772205352783

Пока меня сильно смущает очень низки скор у coursera и средний у последнего запроса

Да, это **нормальная и очень полезная проверка**: она уже показала, что модель как минимум частично работает, но в текущем виде ей нельзя слепо доверять. По документации эта модель вообще обучена как reranker для русскоязычного information retrieval на MS-MARCO Russian passage ranking, то есть она лучше всего работает как второй этап переупорядочивания уже более-менее подходящих кандидатов, а не как универсальный финальный классификатор для грязных, двуязычных и ложно совпадающих сниппетов. [\[52\]](#) [\[53\]](#) [\[54\]](#)

Что видно по твоим результатам

Низкий score у Coursera меня **не смущает**: этот документ действительно больше похож на курс “про аналитику и реляционные базы”, чем на материал именно про MySQL Big Data, так что 0.0019 выглядит логично. Средний score у PostgreSQL-страницы тоже не катастрофа, потому что в тексте есть релевантные слова про Postgres, Spark и Hadoop, но сам фрагмент короткий и не выглядит как прямой “лучший ответ” на запрос. [\[53\]](#) [\[55\]](#)

А вот реально тревожный случай — Big Data at Berkeley с score 0.97. Это почти textbook-пример ложного lexical match: слова Berkeley и Big Data есть, но это не про Berkeley DB, а про студенческую организацию; значит, текущая подача текста модели даёт ей слишком сильную опору на поверхностное совпадение токенов. [\[56\]](#) [\[55\]](#)

Почему так происходит

Cross-encoder читает query и document вместе и выдаёт один score релевантности, но качество сильно зависит от того, **как именно ты сформировал document text**. Если в запросе стоит Berkeley DB Big Data, а в документе есть Berkeley и Big Data, но нет нормального контекста про database/key-value/Oracle/BDB, модель на шумных данных может переоценить такое совпадение, особенно если сниппет короткий или неструктурированный. [\[57\]](#) [\[56\]](#)

Плюс твоя модель русская, а тестовые фрагменты у тебя сейчас в основном английские. Для русских запросов она может быть сильнее, но на смешанном корпусе это уже не идеальный матч. [\[53\]](#)

Что делать прямо сейчас

На этом этапе я бы **не менял пока весь пайплайн**, а сделал две точечные правки входа:

1. В запрос подавать не просто Berkeley DB Big Data, а более структурно:
database: Berkeley DB; topic: Big Data
2. В документ включать явные признаки:
url + snippet, а ещё лучше добавить домен отдельно.
Например:
domain: studentorg.berkeley.edu; url: ...; snippet: ...

Это поможет модели лучше различать технический контент и случайные совпадения по бренду/слову. Для rerankers формат пары очень влияет на итоговый score, потому что модель оценивает именно совместное представление query-document. [\[52\]](#) [\[56\]](#)

Следующий правильный шаг

Я бы сейчас сделал **маленький A/B тест на тех же 5 примерах**:

- вариант А: как у тебя сейчас;
- вариант В: структурированный query и структурированный document.

Например так:

```
pairs = [  
  [  
    "database: Berkeley DB; topic: Big Data",  
    "domain: bd.studentorg.berkeley.edu; url: https://bd.studentorg.berkeley.edu",  
  ],  
  [  
    "database: Berkeley DB; topic: Big Data",  
    "domain: debrabernier.com; url: https://debrabernier.com/navigating-the-de",  
  ]  
]
```

Если после этого `studentorg.berkeley.edu` всё ещё будет выше настоящего Berkeley DB материала, значит, проблема уже не только в формате входа, и тогда мы либо:

- добавим простые rule-based штрафы по домену/ключевым словам,
- либо сравним с cross-encoder/ms-marco-MiniLM-L6-v2, который обучен как сильный MS MARCO reranker и хорошо задокументирован как general-purpose passage reranker. [\[58\]](#) [\[54\]](#)

Мой вывод

Сейчас рано паниковать: Coursera и PostgreSQL выглядят объяснимо, а вот Big Data at Berkeley — это действительно хороший сигнал, что нужен ещё один шаг калибровки входа. Модель уже полезна, но её нужно использовать не “как магический классификатор”, а как скорер внутри более аккуратной схемы. [\[54\]](#) [\[53\]](#)

Давай следующим сообщением я дам тебе **готовый A/B-скрипт**, где мы сравним старый и улучшенный формат входа на этих же примерах.

✱

отлично, давай перейдём к этому этапу

Отлично. Сейчас сделаем **маленький A/B-тест формата входа**, потому что cross-encoder оценивает именно пары текстов, и качество сильно зависит от того, как ты подаёшь query и document.

Документация Sentence Transformers прямо говорит, что `CrossEncoder.predict()` принимает список пар текстов и возвращает score релевантности/сходства для каждой пары. [\[68\]](#) [\[69\]](#)

Что сравниваем

Сравним два варианта:

- А: твой текущий плоский формат;
- В: структурированный формат с явным разделением database, topic, domain, url, snippet. Это полезно, потому что для reranker'ов формат query-document пары влияет на итоговый score, а cross-encoder вообще задуман как второй этап reranking уже найденных кандидатов, а не как “магический классификатор из сырого мусора”.[\[70\]](#) [\[71\]](#)

А/В-скрипт

Запусти этот код как есть:

```
from sentence_transformers import CrossEncoder

model = CrossEncoder(
    "DiTy/cross-encoder-russian-msmarco",
    max_length=512,
    device="cuda"
)

tests = [
    {
        "name": "mysql_good",
        "query_a": "MySQL Big Data",
        "doc_a": "https://dba.stackexchange.com/questions/114539/mysql-big-data-solu",
        "query_b": "database: MySQL; topic: Big Data",
        "doc_b": "domain: dba.stackexchange.com; url: https://dba.stackexchange.com",
    },
    {
        "name": "mysql_course",
        "query_a": "MySQL Big Data",
        "doc_a": "https://www.coursera.org/learn/analytics-mysql Offered by Duke Uni",
        "query_b": "database: MySQL; topic: Big Data",
        "doc_b": "domain: coursera.org; url: https://www.coursera.org/learn/analyti",
    },
    {
        "name": "berkeley_good",
        "query_a": "Berkeley DB Big Data",
        "doc_a": "https://debrabernier.com/navigating-the-depths-oracle-berkeley-db",
        "query_b": "database: Berkeley DB; topic: Big Data",
        "doc_b": "domain: debrabernier.com; url: https://debrabernier.com/navigati",
    },
    {
        "name": "berkeley_false_match",
        "query_a": "Berkeley DB Big Data",
        "doc_a": "https://bd.studentorg.berkeley.edu Big Data at Berkeley is a UC B",
        "query_b": "database: Berkeley DB; topic: Big Data",
        "doc_b": "domain: bd.studentorg.berkeley.edu; url: https://bd.studentorg.b",
    },
    {
        "name": "postgres_good",
        "query_a": "PostgreSQL Big Data",
```

```

        "doc_a": "https://www.enterprisedb.com/blog/powering-big-data-processing-pc
        "query_b": "database: PostgreSQL; topic: Big Data",
        "doc_b": "domain: enterprisedb.com; url: https://www.enterprisedb.com/blog
    }
]

pairs_a = [(t["query_a"], t["doc_a"]) for t in tests]
pairs_b = [(t["query_b"], t["doc_b"]) for t in tests]

scores_a = model.predict(pairs_a)
scores_b = model.predict(pairs_b)

for t, sa, sb in zip(tests, scores_a, scores_b):
    print("=" * 100)
    print("CASE:", t["name"])
    print("A_SCORE:", float(sa))
    print("B_SCORE:", float(sb))
    print("DELTA  :", float(sb - sa))

```

Что смотреть в результате

Нас сейчас интересуют не сами числа, а **изменение порядка**:

- `berkeley_false_match` должен стать ниже `berkeley_good`;
- `mysql_good` должен остаться заметно выше `mysql_course`;
- `postgres_good` желательно немного подрастёт или хотя бы не просядет.

Если улучшенный формат действительно помогает, это будет хорошим признаком, что дальше можно строить пайплайн уже на структурированной подаче текста. [\[69\]](#) [\[71\]](#)

Если ложный Berkeley всё ещё высокий

Тогда мы пойдём на следующий шаг:

- добавим лёгкие rule-based признаки, например штраф за домены типа `studentorg`, `coursera`, `youtube`, `translate`, `pinterest`;
- или сравним эту русскую модель с `cross-encoder/ms-marco-MiniLM-L6-v2`, который часто используют как general-purpose reranker. Это нормально, потому что cross-encoders обычно применяют как reranker поверх кандидатов, а не как полностью автономный фильтр спама. [\[71\]](#) [\[72\]](#) [\[70\]](#)

Пришли вывод A_SCORE / B_SCORE / DELTA, и тогда я скажу, остаётся ли на этой модели и этом формате или уже пора сравнить со второй моделью.

✱

Окей, получились такие результаты:

CASE: mysql_good

A_SCORE: 0.8889395594596863

B_SCORE: 0.7764845490455627

DELTA : -0.11245501041412354

CASE: mysql_course

A_SCORE: 0.0018864325247704983

B_SCORE: 0.016262181103229523

DELTA : 0.014375748112797737

CASE: berkeley_good

A_SCORE: 0.8485031127929688

B_SCORE: 0.8532304763793945

DELTA : 0.004727363586425781

CASE: berkeley_false_match

A_SCORE: 0.9735593795776367

B_SCORE: 0.9170715808868408

DELTA : -0.0564877986907959

CASE: postgres_good

A_SCORE: 0.48734673857688904

B_SCORE: 0.7215616106987

DELTA : 0.2342148721218109

Да, это **хороший** результат: формат В нельзя считать явным победителем. По твоим числам видно, что он сильно помог postgres_good и чуть снизил ложный berkeley_false_match, но не исправил главную проблему до конца.

Что это значит

Это похоже на типичное поведение cross-encoder reranker: формат пары query-document действительно влияет на score, но сам по себе не гарантирует устранение ложных совпадений. Cross-encoders вообще лучше работают как этап reranking для уже найденных кандидатов, а не как полностью автономный классификатор грязных и неоднородных сниппетов. [\[83\]](#) [\[84\]](#) [\[85\]](#)

У тебя это видно очень наглядно: структурирование помогло там, где документ был реально по теме, но почти не “сломало” ложный матч с Berkeley. Это хороший сигнал, что следующая проблема уже не в одном только шаблоне текста, а в сочетании модели, языка корпуса и шумных сниппетов.

Вывод по формату

Я бы **не выбрасывал** идею структурирования совсем, но и не делал её главным решением. Судя по документации и типичному использованию reranker-моделей, полезнее не искать “идеальный prompt-формат”, а сравнить другую модель, особенно если текущая модель специализируется на русском retrieval, а у тебя тестовые примеры во многом англоязычные. [\[86\]](#) [\[87\]](#)

Иными словами:

- формат влияет;
- но проблема Berkeley не решена;
- значит, пора сравнивать **вторую модель** на тех же кейсах.

Следующий шаг

Следующим шагом я бы протестировал cross-encoder/ms-marco-MiniLM-L6-v2 на тех же пяти примерах. Эта модель широко используется как general-purpose MS MARCO cross-encoder reranker, и её логично сравнить с текущей русскоязычной моделью именно на смешанном или англоязычном корпусе. [\[87\]](#) [\[88\]](#)

Лучше всего сделать честное сравнение:

- текущая DiTy/cross-encoder-russian-msmarco;
- cross-encoder/ms-marco-MiniLM-L6-v2;
- один и тот же набор тестов;
- один и тот же формат, лучше пока оставить A как базовый, потому что он уже дал сильный результат на mysql_good и не ухудшал картину искусственно.

Что я предлагаю

Сейчас самый правильный шаг — не городить правила и не запускать весь CSV, а сделать head-to-head **сравнение двух моделей** на этих же 5 кейсах. Если MiniLM опустит berkeley_false_match ниже berkeley_good, то мы возьмём её как основную; если нет, тогда уже подключим лёгкие rule-based штрафы по доменам вроде coursera, youtube, translate, studentorg. [\[85\]](#) [\[88\]](#) [\[87\]](#)

Ниже дам готовый скрипт для сравнения двух моделей на одном наборе примеров.

✱

хорошо, давай переходить к этому этапу

Отлично, идём дальше: теперь делаем head-to-head **сравнение двух cross-encoder моделей** на одном и том же наборе пар. cross-encoder/ms-marco-MiniLM-L6-v2 обучена на задаче MS MARCO Passage Ranking и прямо рекомендуется для information retrieval/reranking, а Sentence Transformers показывает стандартный способ её использования через CrossEncoder(...).predict(pairs). [\[89\]](#) [\[90\]](#) [\[91\]](#)

Что проверяем

Нас сейчас интересуют три вещи:

- опустит ли MiniLM ложный `berkeley_false_match`;
- сохранит ли высокий `SCORE` у `mysql_good`;
- будет ли она в целом стабильнее на англоязычных примерах.

Это логичный шаг, потому что текущая модель у тебя русскоязычная, а тестовый набор сейчас в основном английский, тогда как MS MARCO cross-encoders как раз заточены под web search relevance.^{[92] [93] [89]}

Скрипт сравнения

Запусти такой код:

```
from sentence_transformers import CrossEncoder

tests = [
    {
        "name": "mysql_good",
        "query": "MySQL Big Data",
        "doc": "https://dba.stackexchange.com/questions/114539/mysql-big-data-soluti",
    },
    {
        "name": "mysql_course",
        "query": "MySQL Big Data",
        "doc": "https://www.coursera.org/learn/analytics-mysql Offered by Duke Unive",
    },
    {
        "name": "berkeley_good",
        "query": "Berkeley DB Big Data",
        "doc": "https://debrabernier.com/navigating-the-depths-oracle-berkeley-db-a",
    },
    {
        "name": "berkeley_false_match",
        "query": "Berkeley DB Big Data",
        "doc": "https://bd.studentorg.berkeley.edu Big Data at Berkeley is a UC Ber",
    },
    {
        "name": "postgres_good",
        "query": "PostgreSQL Big Data",
        "doc": "https://www.enterprisedb.com/blog/powering-big-data-processing-pos",
    }
]

pairs = [(t["query"], t["doc"]) for t in tests]

models = {
    "dity_ru": "DiTy/cross-encoder-russian-msmarco",
    "minilm_en": "cross-encoder/ms-marco-MiniLM-L6-v2"
}

all_scores = {}
```



```

for model_name, model_path in models.items():
    print(f"\nLoading: {model_name} -> {model_path}")
    model = CrossEncoder(model_path, max_length=512, device="cuda")
    scores = model.predict(pairs)
    all_scores[model_name] = scores

print("\n" + "=" * 120)
print(f"{'CASE':<24} {'DiTy_RU':>12} {'MiniLM_EN':>12} {'DELTA(EN-RU)':>14}")
print("=" * 120)

for i, t in enumerate(tests):
    ru = float(all_scores["dity_ru"][i])
    en = float(all_scores["minilm_en"][i])
    print(f"{t['name']:<24} {ru:>12.6f} {en:>12.6f} {en - ru:>14.6f}")

```

Как оценивать результат

Идеальный для нас исход такой:

- `mysql_good > mysql_course;`
- `berkeley_good > berkeley_false_match;`
- `postgres_good` не выглядит заниженным.

Именно относительный порядок тут важнее абсолютных чисел, потому что cross-encoders в retrieval обычно используют как scorer/reranker, а не как калиброванный probabilistic classifier. [\[91\]](#) [\[94\]](#) [\[92\]](#)

Что делать после вывода

Если MiniLM покажет лучший порядок на этих пяти кейсах, то берём её как основную модель для следующего шага. Если обе модели всё ещё считают `berkeley_false_match` слишком высоким, тогда уже добавим простой гибрид: score модели плюс rule-based штрафы по домену и ключевым паттернам вроде studentorg, coursera, youtube, translate. [\[95\]](#) [\[92\]](#)

Пришли таблицу со значениями для обеих моделей, и тогда я скажу, какую из них фиксируем как основную перед переходом к прогону по реальным строкам CSV.

✱

Окей, вот такие результаты тестов:

CASE DiTy_RU MiniLM_EN DELTA(EN-RU)

mysql_good 0.888940 7.785728 6.896788
mysql_course 0.001886 -0.741942 -0.743828
berkeley_good 0.848503 7.561782 6.713279
berkeley_false_match 0.973559 6.753084 5.779525
postgres_good 0.487347 4.248654 3.761307

Отлично, теперь уже видно главное: MiniLM_EN выглядит **лучше как ранжировщик**, хотя его числа нельзя напрямую сравнивать с DiTy_RU, потому что MS MARCO cross-encoder возвращает сырые logits, а не обязательно значения от 0 до 1. Документация Sentence Transformers прямо отмечает, что MS MARCO модели могут возвращать logits и что для получения шкалы 0–1 можно загружать их с `activation_fn=torch.nn.Sigmoid()`, при этом ранжирование от этого не меняется. [\[105\]](#) [\[106\]](#) [\[107\]](#)

Что уже можно заключить

По относительному порядку MiniLM_EN ведёт себя разумнее: `mysql_good` выше `mysql_course`, `berkeley_good` выше `berkeley_false_match`, и `postgres_good` выглядит сильнее, чем у русской модели. Это очень хороший знак, потому что именно порядок кандидатов для reranker важнее абсолютного значения score. [\[108\]](#) [\[105\]](#)

У DiTy_RU была критичная ошибка: она ставила `berkeley_false_match` выше настоящего результата про Berkeley DB. У MiniLM_EN эта проблема уже ослаблена, потому что `berkeley_good` поднялся выше ложного совпадения, а это как раз то, что нам и нужно от модели для search relevance. [\[109\]](#) [\[105\]](#)

Какую модель фиксируем

Я бы **фиксировал основной моделью** `cross-encoder/ms-marco-MiniLM-L6-v2`. Она специально обучена для MS MARCO Passage Ranking и в документации Sentence Transformers указана как сильный и достаточно быстрый вариант среди MS MARCO cross-encoders. [\[105\]](#) [\[108\]](#)

Для твоего железа это тоже разумный компромисс: есть модели мощнее, но MiniLM-L6-v2 уже даёт хорошее качество и остаётся достаточно лёгкой для локального прогона. [\[108\]](#)

Что стоит поправить перед реальным CSV

Сейчас лучше сразу перейти на версию с `sigmoid`, чтобы потом было удобнее мыслить порогами и раскладывать строки по полезно / неизвестно / спам. Sentence Transformers прямо рекомендует для MS MARCO моделей использовать `activation_fn=torch.nn.Sigmoid()` если нужны scores в диапазоне 0–1. [\[107\]](#)

Используй такую загрузку:

```
import torch
from sentence_transformers import CrossEncoder

model = CrossEncoder(
    "cross-encoder/ms-marco-MiniLM-L6-v2",
    max_length=512,
    device="cuda",
    activation_fn=torch.nn.Sigmoid()
)
```

После этого score станут удобнее для интерпретации, хотя их смысл всё равно остаётся ranking-oriented, а не “идеальная вероятность класса”.^{[106] [107]}

Следующий шаг

Теперь следующий правильный этап — **не весь CSV сразу**, а маленький реальный прогон на 20–50 строк из твоих файлов. Это нужно, потому что по содержимому CSV у тебя данные шумные, местами склеенные и без чёткого разделения колонок в сниппетах, поэтому надо сначала проверить реальную загрузку строк и scoring на настоящем формате данных.^{[110] [109]}

Я бы делал так:

- читаем один CSV в pandas;
- проверяем реальные названия колонок;
- собираем текст пары из query + db и url + snippet;
- считаем score для первых 20–50 строк;
- смотрим руками, где модель ошибается.

Ниже — что я предлагаю зафиксировать уже сейчас:

Пункт	Решение
Основная модель	cross-encoder/ms-marco-MiniLM-L6-v2 ^{[105] [108]}
API	sentence_transformers.CrossEncoder ^{[106] [111]}
Формат score	C activation_fn=torch.nn.Sigmoid() для шкалы 0–1 ^[107]
Назначение	Reranking / relevance scoring query-document пар, а не “чистая” классификация из коробки ^{[106] [112]}

Следующим сообщением я дам тебе готовый код для чтения одного из твоих CSV и прогона модели по первым строкам с выводом query, url, snippet, score.

✱✱

Отлично, давай тогда для закрепления повторим и ты по простому для начинающего в ml объяснишь, что мы делали на каждом этапе тестов и к какому результату приходили. а после этого уже перейдём к работе с моими CSV

Да — давай закрепим это простым языком. Мы шли не “от теории”, а как в нормальном ML-проекте: брали гипотезу, проверяли её на маленьком наборе примеров и только потом принимали решение, стоит ли идти дальше. [\[123\]](#) [\[124\]](#) [\[125\]](#)

Суть задачи

Твоя задача — не “понять весь сайт”, а оценить, насколько строка из `query + db + url + snippet` реально соответствует запросу, причём в CSV много шумных, обрезанных и ложных совпадений. В самих файлах видно, что там встречаются и хорошие технические страницы, и нерелевантные результаты вроде курсов, агрегаторов, общих статей и ложных совпадений по словам. [\[124\]](#) [\[123\]](#)

Из-за этого мы сразу решили, что задача больше похожа на `relevance scoring / reranking`, чем на генерацию ответа, и поэтому начали не с маленькой LLM, а с готовых `cross-encoder` моделей для оценки пары “запрос-документ”. Для таких моделей стандартный сценарий работы — подать пару текстов и получить `score` релевантности. [\[125\]](#) [\[126\]](#) [\[127\]](#) [\[128\]](#)

Этап 1

Сначала мы выбрали стартовую модель `DiTy/cross-encoder-russian-msmarco` и минимальный стек библиотек вокруг `sentence-transformers`, потому что эта модель уже позиционируется как `CROSS-encoder/reranker` для русскоязычного `retrieval`. Это был не финальный выбор, а первая рабочая гипотеза: “может ли готовая русская модель без обучения вообще отличать хорошие результаты от плохих”. [\[126\]](#) [\[129\]](#) [\[130\]](#)

Простыми словами: на этом этапе мы не пытались добиться идеального качества. Мы просто хотели убедиться, что локальная модель загружается, считает `score` и хотя бы в грубом виде умеет отличать полезный документ от явно слабого. [\[125\]](#) [\[126\]](#)

Этап 2

Потом мы сделали `smoke test` на нескольких примерах руками, а не на всём CSV сразу. Это важный ML-принцип: сначала проверить поведение модели на 5–10 понятных кейсах, чтобы быстро увидеть грубые ошибки. [\[123\]](#) [\[124\]](#)

Результаты были смешанными: `mysql_good` получил высокий `score`, `mysql_course` — очень низкий, что выглядело логично, потому что курс про аналитику и реляционные базы — не лучший ответ на запрос `MySQL Big Data`. Но при этом `Big Data at Berkeley` получил очень высокий `score`, хотя это ложное совпадение и не страница про `Berkeley DB`. [\[124\]](#)

Простыми словами: модель показала, что она **что-то понимает**, но у неё есть опасная слабость — она может переоценивать страницы, где просто совпали слова. Это уже был полезный результат, потому что

мы не стали бы слепо запускать такую модель на весь CSV. [\[124\]](#)

Этап 3

После этого мы проверили вторую гипотезу: может, проблема не только в модели, а ещё и в том, **как мы подаём ей текст**. Поэтому мы сделали A/B-тест двух форматов: обычный плоский текст и более структурированный формат с полями вроде database, topic, domain, url, snippet. [\[131\]](#) [\[132\]](#)

Идея была очень простая: если модель лучше видит структуру входа, ей должно стать легче отличать “Berkeley DB” от просто слова “Berkeley”. Это нормальный приём для reranker-моделей, потому что они оценивают именно пару query-document, и формат подачи влияет на итоговый score. [\[133\]](#) [\[134\]](#) [\[125\]](#)

Результат вышел промежуточным: структурированный формат заметно помог для postgres_good, чуть улучшил berkeley_good и немного опустил ложный berkeley_false_match, но не решил проблему до конца. Значит, формат входа важен, но сам по себе он не спасает от всех ошибок модели. [\[132\]](#) [\[125\]](#)

Простыми словами: мы поняли, что “магического prompt’a” тут нет. Немного помочь можно, но главная проблема сидит глубже — в самой модели и в типе данных. [\[135\]](#) [\[126\]](#)

Этап 4

Дальше мы проверили уже самую важную гипотезу: а не в том ли дело, что первая модель русская, а твои тестовые примеры сейчас в основном англоязычные. Поэтому мы сравнили DiTy/cross-encoder-russian-msmarco с cross-encoder/ms-marco-MiniLM-L6-v2, которая обучена как MS MARCO passage reranker и широко используется именно для information retrieval. [\[127\]](#) [\[136\]](#) [\[126\]](#)

Тут мы смотрели не на абсолютные числа, а на **порядок** результатов. Для reranker это главное: хорошая страница должна быть выше плохой, даже если шкала score у моделей разная. [\[128\]](#) [\[137\]](#)

Сравнение показало, что MiniLM ведёт себя разумнее: mysql_good оказался выше mysql_course, berkeley_good — выше berkeley_false_match, а postgres_good тоже выглядел сильнее, чем у русской модели. Это был уже достаточно уверенный сигнал, что для твоих текущих примеров MiniLM подходит лучше. [\[136\]](#) [\[127\]](#)

Простыми словами: первая модель была “неплохой пробой”, но вторая модель лучше ранжирует именно такие веб-результаты. Значит, её и логично брать как основную. [\[137\]](#) [\[127\]](#)

Что мы зафиксировали

В итоге мы выбрали cross-encoder/ms-marco-MiniLM-L6-v2 как основную модель. Причина простая: по нашим тестам она лучше сохраняет правильный порядок между полезными и ложными страницами, а это для твоей задачи важнее всего. [\[127\]](#) [\[136\]](#)

Ещё мы зафиксировали важную деталь про scores: у MS MARCO cross-encoder моделей значения могут быть сырыми logits, поэтому их не надо воспринимать как “готовую вероятность”. Если нужна более удобная шкала 0–1 для будущих порогов полезно / неизвестно / спам, модель можно загрузить с Sigmoid, и документация Sentence Transformers это прямо допускает. [\[138\]](#) [\[125\]](#)

Если совсем по-простому, то наш путь был таким:

- взяли первую рабочую модель;
- увидели, что она частично работает, но ошибается на ложных совпадениях;
- проверили, помогает ли лучший формат входа;
- увидели, что помогает, но не решает проблему полностью;
- сравнили две модели;
- выбрали ту, которая лучше ранжирует реальные примеры. [\[126\]](#) [\[125\]](#) [\[127\]](#) [\[124\]](#)

Следующим сообщением перейдём к практической части: чтение твоего CSV, проверка колонок и первый прогон модели по реальным строкам.

✱✱

отлично, давай переходить к следующему этапу

Отлично, теперь у нас есть уже нормальная структура: в `brave_url.csv` точно есть колонки `db`, `query`, `url`, `snippet`. Это очень удобно, потому что дальше не нужно ничего парсить вручную из одной слепленной строки — можно сразу собирать пару для модели из этих четырёх полей. [\[139\]](#)

Что делаем сейчас

На этом шаге мы не классифицируем всё подряд, а делаем **маленький реальный прогон** по первым строкам CSV. Цель очень простая: убедиться, что модель нормально работает уже с твоими настоящими данными, а не только с нашими ручными тестами. [\[140\]](#) [\[139\]](#)

Первый реальный скрипт

Запусти такой код на `brave_url.csv`:

```
import pandas as pd
import torch
from sentence_transformers import CrossEncoder

# 1. Загружаем CSV
df = pd.read_csv("brave_url.csv")

# 2. Смотрим базовую информацию
print("COLUMNS:", df.columns.tolist())
print("ROWS:", len(df))
print(df.head(3).to_dict(orient="records"))

# 3. Загружаем выбранную модель
model = CrossEncoder(
    "cross-encoder/ms-marco-MiniLM-L6-v2",
    max_length=512,
    device="cuda",
    activation_fn=torch.nn.Sigmoid()
)

# 4. Берем первые 20 строк
sample = df.head(20).copy()
```

```
# 5. Формируем пары (query, document)
pairs = []
for _, row in sample.iterrows():
    query_text = f"{row['db']} {row['query']}"
    doc_text = f"{row['url']} {row['snippet']}"
    pairs.append((query_text, doc_text))

# 6. Считаем score
scores = model.predict(pairs)

# 7. Сохраняем score и печатаем удобно
sample["score"] = scores

for _, row in sample.iterrows():
    print("=" * 120)
    print("DB      :", row["db"])
    print("QUERY   :", row["query"])
    print("URL      :", row["url"])
    print("SCORE    :", float(row["score"]))
    print("SNIPPET:", str(row["snippet"][:300]))
```

Что мы хотим увидеть

По первым строкам `brave_url.csv` уже видно, что среди результатов по Oracle и Большие данные Oracle есть и очень релевантные страницы Oracle Big Data, и страницы про LOB, размер таблиц, размер БД и другие смежные, но уже не совсем точные темы. Значит, на маленькой выборке должно быть видно, ставит ли модель выше именно страницы про Big Data, а не просто любые страницы, где встречается слово Oracle. ^[139]

Как оценивать вывод

Пока не пытайся сразу придумать точные пороги. Смотри на здравый смысл:

- официальные страницы Oracle про Big Data должны быть высоко;
- страницы про LOB, размеры таблиц и размер БД — скорее средне или ниже;
- совсем случайные совпадения — ещё ниже. ^[139]

Если модель начинает ставить наверх всё подряд только из-за слова Oracle, это сигнал, что потом придётся усиливать схему — например, добавлять более аккуратный текст запроса или rule-based штрафы. Но сначала важно увидеть реальное поведение на 20 строках. ^[141] ^[139]

Что прислать мне дальше

Пришли:

- 10–20 строк вывода со `score`,
- или хотя бы список вида `url + score + короткий комментарий "норм/странно"`.

После этого мы сделаем следующий шаг: добавим в DataFrame колонку с предсказанным классом `useful / unknown / spam` через временные пороги и посмотрим, насколько это похоже на правду.

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://www.oracle.com/europe/big-data/what-is-big-data/>

SCORE : 0.0019650410395115614

SNIPPET: For organizations needing efficient and comprehensive management of big data, the Oracle Cloud Infrastructure (OCI) Big Data platform provides a wide range of capabilities with an exceptional price-to-performance ratio. - странно, при том, что аналогичная статья DB : Oracle

QUERY : Большие данные Oracle

URL : <https://www.oracle.com/cis/big-data/what-is-big-data/?er=228088>

SCORE : 0.9970411658287048

SNIPPET: Если говорить простыми словами, большие данные – **более крупные и сложные**

DB : Oracle

QUERY : Большие данные Oracle

URL : https://sql.ucoz.com/index/razmer_bazy_dannykh/0-126

SCORE : 1.7298980310442857e-05

SNIPPET: **SELECT round((sum(bytes)/1048576/1024),2) "Total(GB)"** FROM

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://www.oracle.com/ru/big-data/guide/what-is-big-data.html>

SCORE : 0.00010517381451791152

SNIPPET: Big data is larger, more complex data sets, especially from new data sources. Learn about the definition and history as well as big data benefits, challenges, and best practices. еще статья от oracle с низким скором

Да, здесь уже видно две важные вещи: SCORE выше 1 для MiniLM **ненормален только если sigmoid реально применился**, потому что документация Sentence Transformers говорит, что с `activation_fn=torch.nn.Sigmoid()` MS MARCO cross-encoder должен возвращать значения в диапазоне 0–1, а без sigmoid модель возвращает сырые logits, которые могут быть примерно от -10 до +10. ^{[142] [143] [144]}

Значит, первое: надо проверить, что ты действительно создал **новый** объект модели именно с `activation_fn=torch.nn.Sigmoid()` и потом именно им делал `predict()`, а не старым объектом без sigmoid. Второе: даже если диапазон исправится, низкие значения у некоторых очень релевантных Oracle-страниц всё равно могут оставаться, потому что cross-encoder — это reranker для пар query-passage, а не идеальный multilingual classifier; у тебя запрос по-русски, а часть сниппетов и URL на английском, что легко ломает оценку. ^{[145] [146] [147]}

Что уже понятно по Oracle

Случай с `oracle.com/cis/...` и русским сниппетом против `oracle.com/europe/...` и английским сниппетом выглядит логично: модель, скорее всего, сильнее реагирует на языковое совпадение запроса и сниппета, чем на сам бренд Oracle. Запрос у тебя Большие данные Oracle, а сниппет Если говорить простыми словами, большие данные... на русском почти дословно совпадает по теме, поэтому такой результат вполне объясним. ^[148]

Низкий score у `oracle.com/ru/...` при английском snippet тоже объясним: домен русский, но сам сниппет англоязычный, и для модели это уже менее “естественная” пара. То есть здесь мы видим не поломку логики, а чувствительность reranker-модели к языку и к конкретному тексту сниппета. ^[147] ^[148]

Что проверить прямо сейчас

Сделай мини-проверку диапазона на двух парах:

```
import torch
from sentence_transformers import CrossEncoder

model = CrossEncoder(
    "cross-encoder/ms-marco-MiniLM-L6-v2",
    max_length=512,
    device="cuda",
    activation_fn=torch.nn.Sigmoid()
)

pairs = [
    ("Oracle Большие данные", "Oracle big data platform and analytics"),
    ("Oracle Большие данные", "котики и рецепты")
]

scores = model.predict(pairs)
print(scores)
print(type(scores[0]), scores.min(), scores.max())
```

Если там снова появятся числа больше 1, значит `sigmoid` у тебя фактически не применяется и надо будет смотреть версию `sentence-transformers` или способ вызова. По документации при активном `sigmoid` score должны быть в диапазоне 0–1. ^[143] ^[142]

Почему результаты всё равно будут “странноваты”

Даже после фикса диапазона надо помнить: `ms-marco-MiniLM-L6-v2` обучалась для `passage ranking` и хорошо работает именно как `scorer` для `query-document` пар, но не обязана давать красивые, интуитивные “вероятности релевантности”. Документация прямо подчёркивает, что для `cross-encoders` важнее ранжирование, чем абсолютное число, и что даже очень похожие тексты не обязаны получать score близкий к 1. ^[149] ^[146] ^[145]

Поэтому нас в реальном CSV будет интересовать не “почему эта хорошая страница получила 0.002”, а:

- выше ли она, чем реально нерелевантные страницы;
- стабильно ли хорошие результаты попадают в верхнюю часть списка;

- можно ли потом отделить полезное от мусора порогами или правилами.^[150] ^[147]

Что я думаю сейчас

Пока у нас есть две рабочие гипотезы:

- либо `sigmoid` реально не сработал и часть “странности” техническая;^[142] ^[143]
- либо даже после его исправления модель будет оставаться чувствительной к языку `русский query` - > `английский snippet`, и тогда нам лучше будет перейти к небольшому **предобработанному запросу**, например `"{db}. {query}"`, и отдельно смотреть на русский и английский наборы.^[148]

Следующий шаг

Сейчас самый правильный порядок такой:

1. Проверить, что `MiniLM` действительно выдаёт 0–1 с `sigmoid`.^[143] ^[142]
2. Если диапазон нормальный — прогнать на первых 20–30 строках снова.
3. После этого уже решить, строим ли простой `baseline`-классификатор по порогам или сначала улучшаем текст пары.^[147] ^[150]

Пришли вывод мини-проверки `scores / min / max`, и тогда я сразу скажу, есть ли у нас техническая проблема с `sigmoid` или уже можно идти дальше к рабочему прогону по CSV.

✱✱

Вот такой вывод получился:

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://www.oracle.com/cis/big-data/what-is-big-data/?er=228088>

SCORE : 0.9970411658287048

SNIPPET: Если говорить простыми словами, большие данные – более крупные и сл

=====

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://www.oracle.com/europe/big-data/what-is-big-data/>

SCORE : 0.0019650410395115614

SNIPPET: For organizations needing efficient and comprehensive management of big data, the Oracle Cloud Infrastructure (OCI) Big Data platform provides a wide range of capabilities with an exceptional price-to-performance ratio.

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://www.oracle.com/cis/big-data/>

SCORE : 0.9993640780448914

SNIPPET: Управление, каталогизация и обработка необработанных данных с помощью Oracle Big Data. Создайте мощное озеро данных, которое будет четко интегрировано в существующие архитектуры и позволит легко подключить данные к пользователям.

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://learn.microsoft.com/ru-ru/dotnet/framework/data/adonet/oracle-lobs>

SCORE : 0.999488115310669

SNIPPET: В следующем примере C# показано, как можно создавать большие объекты (LOB) в таблице Oracle, а затем извлекать и записывать их в виде объектов OracleLob. В примере демонстрируется использование объекта OracleDataReader и методов OracleLob, Read и Write. В этом примере используются типы данных Oracle

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://someit.wordpress.com/2020/10/18/oracle-%D0%BF%D1%80%D0%BE%D1%81%D0%BC%D0%BE%D1%82%D1%80-%D1%80%D0%B0%D0%B7%D0%BC%D0%B5%D1%80%D0%B0-%D0%B1%D0%B4-%D1%82%D0%B0%D0%B1%D0%BB%D0%B8%D1%86%D1%8B-%D0%BA%D0%BE%D0%BB%D0%BE%D0%BD%D0%BA/>

SCORE : 0.003733981167897582

SNIPPET: Работа с размером БД Размер БД в ГБ занимаемых на диске, только файлы с данн

=====

DB : Oracle

QUERY : Большие данные Oracle

URL : https://buran.group/solutions/data_management/bigdata/

SCORE : 0.9950990080833435

**SNIPPET: Oracle Big Data Appliance On Premises:
Программно-аппаратный комплекс в Вашем ЦОД ...
Получайте данные из разных источников.
Собирайте, очищайте и преобразуйте свои данные.**

DB : Oracle

QUERY : Большие данные Oracle

URL : http://apps-oracle.ru/lob_oracle/

SCORE : 0.9980682730674744

SNIPPET: Максимальный размер LOB составляет (4 GB (4 294 967 295 байт) – 1)

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://www.osp.ru/news/articles/2015/26/13046410>

SCORE : 0.9988227486610413

SNIPPET: Big Data Appliance может быть интегрирован с другим программно-аппаратным комплексом – машиной базы данных Oracle Exadata, что позволит демонстрировать сценарии построения аналитических систем, одновременно использующих данные из СУБД Oracle и Hadoop .
29.06.2015 Наталья Дубова Рубрика:Индустрия ...

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://oracle-patches.com/forum/oracle/4285-%D1%80%D0%B0%D0%B7%D0%BC%D0%B5%D1%80-%D0%B1%D0%B0%D0%B7%D1%8B-%D0%B4%D0%B0%D0%BD%D0%BD%D1%8B%D1%85-oracle>

SCORE : 0.903946578502655

SNIPPET: Базы данных: Oracle, MySQL / MariaDb, MS SQL, PostgreSQL, No SQL БД и т.д.

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://habr.com/ru/articles/1033554/>

SCORE : 0.998358428478241

SNIPPET: В базе данных Oracle существует тип данных – **BLOB** (Binary Large OBj

=====

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://nepaday.wordpress.com/2018/02/28/%D0%BD%D0%B0%D0%B9%D1%82%D0%B8-%D1%81%D0%B0%D0%BC%D1%8B%D0%B5-%D0%B1%D0%BE%D0%BB%D1%8C%D1%88%D0%B8%D0%B5-%D1%82%D0%B0%D0%B1%D0%BB%D0%B8%D1%86%D1%8B-%D0%B2-oracle/>

SCORE : 0.005082534160465002

SNIPPET: Топ-10 самых больших (по размеру) таблиц: col owner format a15 col segment_name format a15 SELECT * FROM (SELECT owner, segment_name, bytes/1024/1024 meg FROM dba_segments WHERE segment_type = 'TABLE' ORDER BY bytes/1024/1024 desc) WHERE rownum

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://samag.ru/archive/article/3411>

SCORE : 0.08178114145994186

SNIPPET: Журнал «Системный администратор», читать, скачать, купить

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://logistics.ru/automation/news/issledovaniye-oracle-i-cnews-analytics-bolshie-dannye-prishli-v-rossiyu>

SCORE : 0.9957869648933411

SNIPPET: «Рынок сделал глубокий вдох перед тем, как начать действительно серьезные инвестиции в Big Data, - отмечает Эндрю Сазерленд, старший вице-президент Oracle по технологиям в регионе EMEA. - В Big Data первостепенное значение имеет конечный результат, а не сами технологии. Большие данные позволяют извл

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://habr.com/ru/articles/138270/>

SCORE : 0.9981274008750916

SNIPPET: Совсем недавно на работе стало необходимым контролировать рост размера базы данных Oracle. Связано это было с тем, что места осталось не так много, что -то около 100 Гб, а в двух компаниях (компаниях...)

DB : Oracle

QUERY : Большие данные Oracle

URL : [https://ru.stackoverflow.com/questions/148209](https://ru.stackoverflow.com/questions/1482093/%D0%9A%D0%B0%D0%BA-%D0%B2-oracle-%D0%91%D0%94-%D0%B8%D0%B7%D0%BC%D0%B5%D1%80%D0%B8%D1%82%D1%8C-%D1%80%D0%B0%D0%B7%D0%BC%D0%B5%D1%80-%D0%BE%D0%B1%D1%8A%D0%B5%D0%BC-%D0%B4%D0%B0%D0%BD%D0%BD%D1%8B%D1%85-%D0%B2-%D0%B2%D1%8B%D0%B1%D0%BE%D1%80%D0%BA%D0%B5-%D1%81%D0%B5%D0%BB%D0%B5%D0%BA%D1%82%D0%B5)

[3/%D0%9A%D0%B0%D0%BA-%D0%B2-oracle-%D0%91%D0%94-%D0%B8%D0%B7%D0%BC%D0%B5%D1%80%D0%B8%D1%82%D1%8C-%D1%80%D0%B0%D0%B7%D0%BC%D0%B5%D1%80-%D0%BE%D0%B1%D1%8A%D0%B5%D0%BC-%D0%B4%D0%B0%D0%BD%D0%BD%D1%8B%D1%85-%D0%B2-%D0%B2%D1%8B%D0%B1%D0%BE%D1%80%D0%BA%D0%B5-%D1%81%D0%B5%D0%BB%D0%B5%D0%BA%D1%82%D0%B5](https://ru.stackoverflow.com/questions/1482093/%D0%9A%D0%B0%D0%BA-%D0%B2-oracle-%D0%91%D0%94-%D0%B8%D0%B7%D0%BC%D0%B5%D1%80%D0%B8%D1%82%D1%8C-%D1%80%D0%B0%D0%B7%D0%BC%D0%B5%D1%80-%D0%BE%D0%B1%D1%8A%D0%B5%D0%BC-%D0%B4%D0%B0%D0%BD%D0%BD%D1%8B%D1%85-%D0%B2-%D0%B2%D1%8B%D0%B1%D0%BE%D1%80%D0%BA%D0%B5-%D1%81%D0%B5%D0%BB%D0%B5%D0%BA%D1%82%D0%B5)

SCORE : 0.1099138930439949

SNIPPET: При селекте в таблице в БД Oracle выводятся различные столбцы с разными типами данных, в основном varchar2, number,date,rowid. Как правильно установить размер в Мб определенной ...

DB : Oracle

QUERY : Большие данные Oracle

URL : <http://torofimofu.blogspot.com/2015/05/oracle.html?m=1>

SCORE : 0.9953565001487732

SNIPPET: По сути, большой объект - это абстрактный тип для манипуляции данны

=====

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://sky.pro/wiki/sql/zapusk-bolshikh-sql-faylov-v-v-oracle-import-bazy-dannykh/>

SCORE : 0.998996913433075

SNIPPET: Больше 55 · Для того чтобы осуществить импорт файла .sql в Oracle SQL Developer, пройдите следующие шаги: Подключитесь к требуемой базе данных. Откройте файл .sql, выбрав Файл > Открыть или просто перенесите его в рабочую область.

DB : Oracle

QUERY : Большие данные Oracle

URL : https://sql.ucoz.com/index/razmer_bazy_dannykh/0-126

SCORE : 1.7298980310442857e-05

SNIPPET: SELECT round((sum(bytes)/1048576/1024),2) "Total(GB)" FROM

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://oracle-patches.com/oracle/prof/%D1%82%D0%B0%D0%B1%D0%BB%D0%B8%D1%86%D1%8>

B-oracle

SCORE : 0.9974340796470642

SNIPPET: Кластеризованная таблица является частью группы таблиц, которые разделяют между собой одни и те же блоки данных, поскольку данные из кластеризованных таблиц часто запрашиваются совместно. Секционированные таблицы. Секционированная таблица позволяет делить большие объемы данных на подтаблицы, именуем

DB : Oracle

QUERY : Большие данные Oracle

URL : <https://www.oracle.com/ru/big-data/guide/what-is-big-data.html>

SCORE : 0.00010517381451791152

SNIPPET: Big data is larger, more complex data sets, especially from new data sources. Learn about the definition and history as well as big data benefits, challenges, and best practices.

И пока выброс за единицу только в 1 случае, не знаю, как это влияет, в остальных случаях сигмоида работает

Да, теперь картина уже понятнее: проблема **не в выбросе выше 1**, а в том, что модель как ранжировщик на твоих данных ведёт себя нестабильно по смыслу. Документация Sentence Transformers прямо говорит, что такие MS MARCO cross-encoders обучены для **re-ranking query-passage пар** и выдают relevance score, а не полноценную семантическую классификацию по всем случаям; особенно это важно, когда запрос и документ смешаны по языкам или когда в документе есть сильные поверхностные совпадения по словам. ^[158] ^[159] ^[160]

Что мы видим по факту

Хорошая новость: модель реально умеет поднимать наверх явные страницы про Oracle Big Data, например официальные Oracle Big Data страницы и новости про Big Data Appliance. Плохая новость: она также очень высоко оценивает страницы про LOB, BLOB, размер БД, большие таблицы и импорт больших SQL-файлов, то есть цепляется за слово “большие” и за Oracle, хотя это уже другая тема. ^[161]

Это значит, что для твоей задачи **голый cross-encoder как единственный классификатор не годится**. Он полезен как `scorer`, но если оставить только его, то “большие данные Oracle” и “большие объёмы данных/LOB в Oracle” будут часто смешиваться.^{[160] [161]}

Простое объяснение проблемы

Запрос Большие данные Oracle для модели не такой точный, как нам кажется человеку. Она видит:

- Oracle;
- большие;
- данные.

И страницы про BLOB, LOB, “большие таблицы”, “большой SQL-файл” или “большой объём данных” для неё выглядят достаточно близко, потому что по словам это реально рядом, хотя по смыслу это не обязательно Big Data как отдельная технологическая тема.^[161]

Проще говоря: модель пока плохо различает:

- Big Data как домен/технологию;
- large data / large objects / big tables как просто большие объёмы.
Это уже очень важное открытие, и именно ради этого мы и делали маленькие тесты до запуска на весь CSV.^[161]

К какому выводу мы пришли

На этом этапе можно честно зафиксировать:

- модель MiniLM лучше предыдущей как reranker;^{[159] [162]}
- но для твоих данных её нельзя использовать напрямую как готовый 3-классовый классификатор;^{[158] [160]}
- значит, нужен **гибридный baseline**: немного правил + score модели.^[161]

Это нормальная практика. В реальных IR/NLP задачах очень часто не хватает одной модели, особенно если данные шумные и задача зависит от тонкой разницы между темами.^{[163] [160]}

Что делать дальше

Я бы теперь перешёл к **правилу-предфильтру** перед моделью. Идея такая:

- если в `snippet/url` есть сильные признаки настоящего Big Data (`hadoop`, `spark`, `data lake`, `analytics`, `big data appliance`, `hdfs`, `etl`, `warehouse`) — это плюс;
- если есть признаки “не той темы” (`LOB`, `BLOB`, `CLOB`, `размер БД`, `размер таблиц`, `import sql`, `sql file`, `chunk`, `dba_data_files`) — это минус;
- потом уже score модели используется как дополнительный фактор.^[161]

То есть схема станет такой:

1. Собираем признаки из `url + snippet`.
2. Считаем rule-based бонус/штраф.

3. Считаем score модели.

4. По комбинации решаем:

- полезно,
- неизвестно,
- спам.

Почему это лучше

Потому что твои данные очень явно показывают тематический сдвиг. Страница про Oracle Big Data Appliance — это по теме, а страница про LOB в Oracle — нет, даже если там есть слова “большие” и “данные”. Модель одну эту грань держит плохо, а простые словарные правила здесь как раз очень сильны. [\[161\]](#)

Следующий практический шаг

Теперь я бы уже не мучил одну только модель, а сделал первый **рабочий baseline-классификатор** для CSV:

- читаем строки;
- считаем model_score;
- считаем rule_score;
- выводим итоговый класс.

Это будет намного полезнее, чем ещё долго сравнивать модели. По сути, мы уже получили главный результат исследования: **нужен гибрид, а не чистый cross-encoder**. [\[160\]](#) [\[161\]](#)

Следующим сообщением я могу дать тебе готовый Python-код для brave_url.csv, который:

- добавляет признаки big_data_positive / offtopic_large_data;
- считает score модели;
- присваивает классы useful / unknown / spam;
- выводит первые результаты для ручной проверки.

✱✱

отлично, давай переходить к этому шагу

Да, теперь логично перейти к **гибридному baseline**: у тебя уже видно, что модель поднимает и хорошие Oracle Big Data страницы, и оффтопик про LOB/BLOB, размер БД и “большие” SQL-файлы, поэтому одного model score недостаточно. Cross-encoder здесь полезен как reranker для пары query-document, но не как готовый финальный классификатор без дополнительных правил. [\[175\]](#) [\[176\]](#) [\[177\]](#)

Что делаем

Сейчас мы добавим к score модели простой rule-based слой. Идея такая: слова и фразы про big data, hadoop, spark, data lake, appliance, analytics будут давать плюс, а lob, blob, размер бд, размер таблиц, sql file, import и похожие оффтопик-сигналы — минус. ^[175]

Код baseline

```
import re
import pandas as pd
import torch
from sentence_transformers import CrossEncoder

# ===== 1) Загрузка данных =====
df = pd.read_csv("brave_url.csv")

# При желании потом поменяешь на yandex_url.csv
# df = pd.read_csv("yandex_url.csv")

# На всякий случай нормализуем названия колонок
df.columns = [c.strip().lower() for c in df.columns]

required = ["db", "query", "url", "snippet"]
missing = [c for c in required if c not in df.columns]
if missing:
    raise ValueError(f"Нет колонок: {missing}. Есть: {df.columns.tolist()}")

# ===== 2) Модель =====
model = CrossEncoder(
    "cross-encoder/ms-marco-MiniLM-L6-v2",
    max_length=512,
    device="cuda",
    activation_fn=torch.nn.Sigmoid()
)

# ===== 3) Словари правил =====
POSITIVE_TERMS = [
    "big data", "bigdata", "большие данные",
    "hadoop", "spark", "hdfs", "data lake", "озеро данных",
    "data warehouse", "etl", "analytics", "аналитик",
    "big data appliance", "oracle big data appliance",
    "apache hive", "apache kafka", "nosql", "distributed", "распредел"
]

NEGATIVE_TERMS = [
    "lob", "blob", "clob", "nclob", "bfile",
    "размер бд", "размер базы", "размер таблиц", "largest tables",
    "dba_data_files", "v$datafile", "v$datafile",
    "sql file", ".sql", "import sql", "oraclelob",
    "chunk parameter", "oracle lob", "просмотр размера",
    "найти самые большие таблицы", "импорт файла"
]

BAD_DOMAINS = [
    "coursera.org", "youtube.com", "youtu.be", "pinterest.com",
```

```

    "translate", "studentorg", "calendar-uk.co.uk"
]

GOOD_DOMAINS_HINT = [
    "oracle.com", "enterprisedb.com", "postgresql.org",
    "percona.com", "cybertec-postgresql.com"
]

def norm_text(x):
    x = "" if pd.isna(x) else str(x)
    x = x.lower()
    x = re.sub(r"<[^>]+>", " ", x)
    x = re.sub(r"\s+", " ", x).strip()
    return x

def count_terms(text, terms):
    return sum(1 for t in terms if t in text)

def rule_score(url, snippet):
    text = f"{url} {snippet}"
    text = norm_text(text)

    pos = count_terms(text, POSITIVE_TERMS)
    neg = count_terms(text, NEGATIVE_TERMS)
    bad_domain = any(d in text for d in BAD_DOMAINS)
    good_domain = any(d in text for d in GOOD_DOMAINS_HINT)

    score = 0.0
    score += pos * 0.18
    score -= neg * 0.22

    if good_domain:
        score += 0.08
    if bad_domain:
        score -= 0.20

    return max(-1.0, min(1.0, score)), pos, neg, good_domain, bad_domain

# ===== 4) Подготовка пар для модели =====
pairs = []
for _, row in df.iterrows():
    query_text = f"{row['db']} {row['query']}"
    doc_text = f"{row['url']} {row['snippet']}"
    pairs.append((query_text, doc_text))

# ===== 5) Score модели =====
model_scores = model.predict(pairs, batch_size=32, show_progress_bar=True)

# На всякий случай подрежем в [0, 1]
model_scores = [float(max(0.0, min(1.0, s))) for s in model_scores]

# ===== 6) Rule-based признаки =====
rule_scores = []
pos_hits = []
neg_hits = []
good_domain_flags = []

```

```

bad_domain_flags = []

for _, row in df.iterrows():
    rs, ph, nh, gd, bd = rule_score(row["url"], row["snippet"])
    rule_scores.append(rs)
    pos_hits.append(ph)
    neg_hits.append(nh)
    good_domain_flags.append(gd)
    bad_domain_flags.append(bd)

df["model_score"] = model_scores
df["rule_score"] = rule_scores
df["positive_hits"] = pos_hits
df["negative_hits"] = neg_hits
df["good_domain"] = good_domain_flags
df["bad_domain"] = bad_domain_flags

# ===== 7) Комбинированный score =====
# model_score: 0..1
# rule_score: -1..1 -> переведем в 0..1
df["rule_score_01"] = (df["rule_score"] + 1) / 2

# Можно менять веса
df["final_score"] = 0.7 * df["model_score"] + 0.3 * df["rule_score_01"]

# ===== 8) Классы =====
def assign_label(row):
    fs = row["final_score"]
    neg = row["negative_hits"]
    pos = row["positive_hits"]

    # жесткий оффтопик
    if neg >= 2 and pos == 0 and fs < 0.55:
        return "spam"

    if fs >= 0.72:
        return "useful"
    elif fs >= 0.45:
        return "unknown"
    else:
        return "spam"

df["label"] = df.apply(assign_label, axis=1)

# ===== 9) Сохраняем =====
df.to_csv("brave_scored.csv", index=False, encoding="utf-8-sig")

# ===== 10) Просмотр первых результатов =====
preview_cols = [
    "db", "query", "url", "model_score", "rule_score",
    "final_score", "positive_hits", "negative_hits", "label"
]

print(df[preview_cols].head(30).to_string(index=False))

```


Как это работает

Здесь `model_score` отвечает за “общее ощущение релевантности”, а `rule_score` — за тематическую поправку, чтобы отделять Big Data от “больших объектов”, “больших таблиц” и похожих ложных совпадений. По твоим Oracle-примерам именно эта путаница и была главной проблемой. ^[175]

Классы пока временные: `useful` для явно хороших, `spam` для явно мимо, `unknown` для всего пограничного. Это нормально для первого baseline, потому что сейчас важнее получить рабочую фильтрацию, чем сразу идеально настроить все пороги. ^[177]

Что смотреть после запуска

Сначала посмотри на 20–30 первых строк и проверь, стали ли страницы про LOB, BLOB, размер БД и импорт SQL чаще уходить в `spam` или хотя бы в `unknown`, а страницы реально про Oracle Big Data — в `useful`. По твоим прошлым примерам именно это будет главным признаком, что гибридная схема работает лучше, чем одна модель. ^[175]

После этого пришли:

- 10–20 строк из `brave_scored.csv`,
- особенно те, где `label` кажется странным,
- и желательно 2–3 хороших и 2–3 плохих примера.

Тогда следующим шагом мы уже спокойно подкрутим словари и пороги под твой реальный датасет.

✱✱

Oracle, Большие данные Oracle, <https://www.oracle.com/europe/big-data/what-is-big-data/>, For organizations needing efficient and comprehensive management of big data, the Oracle Cloud Infrastructure (OCI) Big Data platform provides a wide range of capabilities with an exceptional price-to-performance ratio.,0.0019650410395115614,0.26,1,0,True,False,0.63,0.1903755287276581,spam - все еще спам, остается проблема с тем, что запросе на русском и ответе на английском у нас низкий скор

Oracle, Большие данные Oracle, <https://www.oracle.com/europe/big-data/what-is-big-data/>, For organizations needing efficient and comprehensive management of big data, the Oracle Cloud Infrastructure (OCI) Big Data platform provides a wide range of capabilities with an exceptional price-to-performance ratio., 0.0019650410395115614, 0.26, 1, 0, True, False, 0.63, 0.1903755287276581, spam - та же проблема. также я быстренько проанализировал ситуация в общем по меткам:

=====

=====

BRAVE RESULTS:

Всего URL: 2826

Спам URL: 638

Доля спама: 22.58%

Полезные URL: 1870

Доля полезных URL: 66.17%

YANDEX RESULTS:

Всего URL: 3067

Спам URL: 881

Доля спама: 28.73%

Полезные URL: 1888

Доля полезных URL: 61.56%

И меня настораживает высокая доля спама из-за типовых ошибок в примерах, которые я тебе скинул. Если нужно - могу скинуть тебе целиком brave.csv, чтобы ты больше понял проблему. Как одно из костыльных решений - можно подключить какуюнибудь модель-переводчик, чтобы определять язык запроса и сниппета и переводить их в 1 язык

Да, ты правильно заметил две реальные проблемы: mixed-language пары действительно ломают model_score, а высокая доля useful при таких типовых ошибках означает, что текущие пороги и правила ещё слишком сырые. По твоим примерам видно, что английская официальная Oracle Big Data

страница получает почти нулевой score только из-за несоответствия языка, хотя по смыслу она хорошая, а значит сейчас модель наказывает не релевантность, а языковую форму. ^[188]

Что это значит

Это не повод сразу тащить полноценный переводчик в пайплайн. Для baseline проще, быстрее и надёжнее сначала сделать **language-aware rule layer**, потому что в CSV уже видны сильные текстовые маркеры `big data`, `oracle big data`, `hadoop`, `spark`, `data lake`, а также оффтопик-маркеры `lob`, `blob`, размер бд, таблицы, `sql file`; эти признаки работают даже без перевода. ^[188]

Полноценный машинный перевод — уже второй шаг, если простая нормализация не даст эффекта. На твоём железе это возможно, но сначала лучше выжать максимум из дешёвых улучшений, потому что перевод добавит задержку и усложнит весь pipeline. ^{[189] [190]}

Что я предлагаю

Сейчас лучший следующий шаг — не “переводить всё”, а:

- распознавать, что `query` на русском, а `snippet` на английском;
- добавлять **language bonus**, если официальная/хорошая страница содержит сильные английские Big Data термины;
- отдельно расширить positive/negative словари;
- немного ослабить зависимость от `model_score`. ^[188]

То есть если строка выглядит так:

- `oracle.com`,
- в URL есть `/big-data/`,
- в `snippet` есть `big data`, `OCI`, `data platform`,
- а `query` русская,
то такую строку нельзя валить в `spam` только из-за низкого cross-encoder score. ^[188]

Почему это лучше перевода на этом этапе

Потому что у тебя проблема не просто “русский vs английский”, а ещё и **тематическая путаница** между:

- `real Big Data`,
- `large object`,
- `big tables`,
- `size of database`. ^[188]

Перевод выравнивает язык, но не решит автоматически путаницу Big Data vs BLOB/LOB. А вот хорошие rule-based признаки бьют именно в корень проблемы. ^[188]

Обновлённая идея baseline

Я бы сейчас сделал 3 улучшения сразу:

- Добавил позитивные паттерны в URL:
`/big-data/, big-data-sql, big-data-appliance, datawarehouse-bigdata, big-data-service.`
[188]
- Усилил негативные паттерны:
`lob, blob, clob, размер, table size, largest tables, sql file, import, dba_data_files, v$datafile.`
[188]
- Добавил правило mixed-language rescue:
если query кириллическая, snippet латиницей, но в URL/snippet есть сильные Big Data термы и нет оффтопик-маркеров, то поднимать класс хотя бы до unknown, а иногда и до useful.
[188]

По поводу полного CSV

Да, целиком `brave_url.csv` присылать не нужно — он уже у меня есть среди вложений, и по содержимому видно достаточно, чтобы понять типичные ошибки: official Big Data pages, новости, документация, курсы, форумы, size/LOB страницы и ложные совпадения вроде Big Data at Berkeley.
[188]

Что делаем дальше

Я бы сейчас не подключал переводчик, а сначала дал тебе **улучшенную версию baseline-кода** с:

- language-aware rescue,
- URL-паттернами,
- более жёстким штрафом за LOB/BLOB/size,
- новыми порогами.
[188]

Если после этого хорошие английские страницы всё ещё будут тонуть, тогда уже можно будет аккуратно добавить лёгкий переводческий слой только для query, а не переводить весь CSV. Хочешь, я сейчас сразу дам тебе вторую версию Python-кода именно под эту улучшенную схему?

✱✱

давай

Да — вот вторая версия baseline, уже с поправкой на смешанный язык и с более сильным разделением между настоящим Big Data и оффтопиком вроде LOB/BLOB/размер БД. Она нужна именно потому, что в твоих данных есть хорошие англоязычные Oracle Big Data страницы, которые сейчас проваливаются из-за русского запроса, и одновременно есть много ложных совпадений по словам “большие” и “данные”.
[191] [192] [193]

Что меняем

Теперь мы делаем четыре вещи: уменьшаем вес `model_score`, усиливаем URL и keyword-сигналы, добавляем простое определение языка и вводим `mixed-language rescue`, если страница явно про Big Data, но сниппет на другом языке. По твоим примерам это должно помочь официальным Oracle Big Data страницам и при этом не пускать наверх страницы про LOB, BLOB, sql file и размер базы.^[191]

Код

```
import re
import pandas as pd
import torch
from sentence_transformers import CrossEncoder

# =====
# 1) Загрузка
# =====
df = pd.read_csv("brave_url.csv")
# df = pd.read_csv("yandex_url.csv")

df.columns = [c.strip().lower() for c in df.columns]

required = ["db", "query", "url", "snippet"]
missing = [c for c in required if c not in df.columns]
if missing:
    raise ValueError(f"Нет колонок: {missing}. Есть: {df.columns.tolist()}")

# =====
# 2) Модель
# =====
model = CrossEncoder(
    "cross-encoder/ms-marco-MiniLM-L6-v2",
    max_length=512,
    device="cuda",
    activation_fn=torch.nn.Sigmoid()
)

# =====
# 3) Словари
# =====
POSITIVE_TERMS = [
    "big data", "bigdata", "большие данные",
    "data lake", "озеро данных",
    "hadoop", "spark", "hdfs", "hive", "kafka", "hbase",
    "analytics", "analytic", "аналитик", "анализ данных",
    "data warehouse", "warehouse",
    "etl", "elt",
    "nosql", "distributed", "распредел",
    "oci big data", "oracle big data",
    "big data appliance", "big data service", "big data sql",
    "data platform", "massive volumes of data"
]

NEGATIVE_TERMS = [
    "lob", "blob", "clob", "nclob", "bfile", "oraclelob",
```

```

    "large object", "binary large object",
    "размер бд", "размер базы", "размер таблиц", "размер таблицы",
    "largest tables", "big table", "большие таблицы",
    "sql file", ".sql", "import sql", "импорт файла", "sql developer",
    "chunk parameter", "dba_data_files", "dba_segments", "v$datafile",
    "просмотр размера", "найти самые большие таблицы",
    "datatype limits", "limits for datatypes",
    "табличное пространство", "tablespace"
]

```

```

POSITIVE_URL_PATTERNS = [
    "/big-data/",
    "big-data-sql",
    "big-data-appliance",
    "big-data-service",
    "datawarehouse-bigdata",
    "big-data-cloud",
    "big-data-discovery",
    "iaas/bigdata",
    "/bigdata/"
]

```

```

NEGATIVE_URL_PATTERNS = [
    "oracle-lobs",
    "lob_oracle",
    "razmer",
    "razmer-bazy",
    "razmer-tabl",
    "blob",
    "clob",
    "sql-fayl",
    "import-bazy",
    "datatype-limits"
]

```

```

GOOD_DOMAINS = [
    "oracle.com",
    "docs.oracle.com",
    "cloud.oracle.com",
    "oci.oraclecloud.com",
    "enterprisedb.com",
    "postgresql.org",
    "percona.com",
    "cybertec-postgresql.com"
]

```

```

BAD_DOMAINS = [
    "youtube.com",
    "youtu.be",
    "pinterest.com",
    "coursera.org",
    "classcentral.com"
]

```

```

# =====
# 4) Вспомогательные функции

```

```

# =====
def clean_text(x):
    x = "" if pd.isna(x) else str(x)
    x = x.lower()
    x = re.sub(r"<[^>]+>", " ", x)
    x = re.sub(r"\s+", " ", x).strip()
    return x

def contains_cyrillic(text):
    return bool(re.search(r"[а-яё]", text.lower()))

def contains_latin(text):
    return bool(re.search(r"[a-z]", text.lower()))

def detect_lang_simple(text):
    text = clean_text(text)
    cyr = len(re.findall(r"[а-яё]", text))
    lat = len(re.findall(r"[a-z]", text))
    if cyr > lat * 1.3 and cyr >= 3:
        return "ru"
    if lat > cyr * 1.3 and lat >= 3:
        return "en"
    return "mixed"

def count_terms(text, terms):
    return sum(1 for t in terms if t in text)

def count_url_patterns(url, patterns):
    url = clean_text(url)
    return sum(1 for p in patterns if p in url)

def is_good_domain(url):
    url = clean_text(url)
    return any(d in url for d in GOOD_DOMAINS)

def is_bad_domain(url):
    url = clean_text(url)
    return any(d in url for d in BAD_DOMAINS)

def compute_rule_features(row):
    url = clean_text(row["url"])
    snippet = clean_text(row["snippet"])
    query = clean_text(row["query"])
    db = clean_text(row["db"])

    full_text = f"{db} {query} {url} {snippet}"

    pos_hits = count_terms(full_text, POSITIVE_TERMS)
    neg_hits = count_terms(full_text, NEGATIVE_TERMS)

    pos_url_hits = count_url_patterns(url, POSITIVE_URL_PATTERNS)
    neg_url_hits = count_url_patterns(url, NEGATIVE_URL_PATTERNS)

    good_domain = is_good_domain(url)
    bad_domain = is_bad_domain(url)

```

```

query_lang = detect_lang_simple(query)
snippet_lang = detect_lang_simple(snippet)
mixed_lang = query_lang != snippet_lang and "mixed" not in (query_lang, snippet_

score = 0.0

# Базовые сигналы
score += pos_hits * 0.16
score -= neg_hits * 0.28

score += pos_url_hits * 0.22
score -= neg_url_hits * 0.30

if good_domain:
    score += 0.10
if bad_domain:
    score -= 0.15

# Особое усиление: official big data page
if good_domain and (pos_url_hits >= 1 or pos_hits >= 2):
    score += 0.18

# Если mixed-language, но strong big data сигналы есть, а оффтопика нет – спасаем
rescue_flag = False
if mixed_lang and good_domain and (pos_url_hits >= 1 or pos_hits >= 2) and neg_h
    score += 0.28
    rescue_flag = True

# Если mixed-language, но при этом LOB/BLOB/size – не спасаем
if mixed_lang and (neg_hits >= 1 or neg_url_hits >= 1):
    score -= 0.08

score = max(-1.0, min(1.0, score))

return {
    "rule_score": score,
    "pos_hits": pos_hits,
    "neg_hits": neg_hits,
    "pos_url_hits": pos_url_hits,
    "neg_url_hits": neg_url_hits,
    "good_domain": good_domain,
    "bad_domain": bad_domain,
    "query_lang": query_lang,
    "snippet_lang": snippet_lang,
    "mixed_lang": mixed_lang,
    "rescue_flag": rescue_flag
}

```

```

# =====
# 5) Пары для модели
# =====

```

```

pairs = []
for _, row in df.iterrows():
    query_text = f"{row['db']} | {row['query']}"
    doc_text = f"{row['url']} | {row['snippet']}"
    pairs.append((query_text, doc_text))

```



```

model_scores = model.predict(pairs, batch_size=32, show_progress_bar=True)
model_scores = [float(max(0.0, min(1.0, s))) for s in model_scores]

df["model_score"] = model_scores

# =====
# 6) Rule features
# =====
features = df.apply(compute_rule_features, axis=1, result_type="expand")
df = pd.concat([df, features], axis=1)

# rule_score: [-1, 1] -> [0, 1]
df["rule_score_01"] = (df["rule_score"] + 1.0) / 2.0

# =====
# 7) Финальный score
# =====
# Уменьшаем роль модели, потому что mixed-language пары ломают scorer
df["final_score"] = (
    0.45 * df["model_score"] +
    0.55 * df["rule_score_01"]
)

# =====
# 8) Итоговая метка
# =====
def assign_label(row):
    fs = row["final_score"]
    neg = row["neg_hits"]
    pos = row["pos_hits"]
    pos_url = row["pos_url_hits"]
    neg_url = row["neg_url_hits"]
    rescue = row["rescue_flag"]
    good_domain = row["good_domain"]

    # Жесткий оффтопик
    if (neg + neg_url) >= 2 and (pos + pos_url) == 0:
        return "spam"

    # LOB/BLOB/size темы обычно не Big Data
    if neg >= 1 and neg_url >= 1 and fs < 0.70:
        return "spam"

    # Mixed-language rescue для official big data страниц
    if rescue and good_domain and (pos + pos_url) >= 2 and fs >= 0.52:
        return "useful"

    if fs >= 0.68:
        return "useful"
    elif fs >= 0.44:
        return "unknown"
    else:
        return "spam"

df["label"] = df.apply(assign_label, axis=1)

```

```

# =====
# 9) Сохранение
# =====
out_cols = [
    "db", "query", "url", "snippet",
    "model_score", "rule_score", "rule_score_01", "final_score",
    "pos_hits", "neg_hits", "pos_url_hits", "neg_url_hits",
    "good_domain", "bad_domain",
    "query_lang", "snippet_lang", "mixed_lang", "rescue_flag",
    "label"
]

df[out_cols].to_csv("brave_scored_v2.csv", index=False, encoding="utf-8-sig")

# =====
# 10) Быстрая проверка
# =====
preview = df[out_cols].head(40)
print(preview.to_string(index=False))

print("\nРаспределение классов:")
print(df["label"].value_counts(dropna=False))

print("\nДоли классов:")
print((df["label"].value_counts(normalize=True, dropna=False) * 100).round(2))

```

Что должно улучшиться

Для строки вроде `oracle.com/europe/big-data/what-is-big-data/` теперь должен сработать не только слабый `model_score`, но и хороший домен, `positive URL pattern /big-data/`, `positive keywords big data` и `rescue` для пары “русский `query` + английский `snippet`”. Это как раз тот случай, который в твоих прошлых результатах падал в `spam`, хотя по смыслу он хороший. ^[191]

При этом страницы про `LOB`, `BLOB`, размер таблиц, размер БД и импорт `.sql` должны чаще уходить вниз, потому что теперь у них сильнее `negative term penalty` и `negative URL penalty`. В твоём CSV таких оффтопик-примеров много, и именно они показывали, что одной модели недостаточно. ^[191]

Как проверить

Сначала посмотри именно на те проблемные Oracle-строки, которые ты уже присылал: `official Big Data pages`, `LOB/BLOB`, размер БД, большие таблицы, `import SQL`. Если `official Big Data` страницы ушли хотя бы в `unknown/useful`, а `LOB/BLOB/size` чаще стали `spam/unknown`, значит направление правильное. ^[191]

Потом посмотри на общее распределение меток: если `useful` всё ещё подозрительно много, следующим шагом мы сделаем маленькую ручную валидацию на 50–100 строк и подгоним пороги уже не “на глаз”, а по мини-разметке. Это будет надёжнее, чем сразу подключать переводчик ко всему пайплайну. ^{[193] [191]}

так, вот обновлённый датасет уже со скорями. У нас получились итоговые оценки:

Распределение классов:

```
label
useful 1942
spam 456
unknown 428
Name: count, dtype: int64
```

Доли классов:

```
label
useful 68.72
spam 16.14
unknown 15.15
Name: proportion, dtype: float64
```

Конечно, есть к чему стремиться, но это уже лучше, чем было. Как думаешь - оставляем как есть, или всеми правдами и неправдами пытаемся докрутить долю `useful` до хотя бы 80 процентов?

Я бы **не** пытался любой ценой дотянуть `useful` до 80%. Сейчас важнее не доля `useful`, а то, насколько чистым и осмысленным получается этот класс на реальных примерах.

Почему не 80%

Сам по себе высокий процент `useful` ничего не гарантирует: его можно искусственно поднять просто ослабив пороги, но тогда в `useful` начнут массово попадать пограничные и мусорные страницы. В твоём `brave_scored_v2.csv` уже видно, что в `useful` проскакивают спорные примеры вроде Laracasts-дискуссии про большие датасеты в Redis, Google Groups-треда про `threshold` для `data set` и общих `glossary/marketing`-страниц про Redis, которые не выглядят как уверенно полезные ответы на запрос `Big Data Redis`.^[194]

Что уже видно

У тебя есть не только обычные ложные срабатывания, но и системная проблема с совпадением по словам вместо смысла. Например, для `exist Big Data` в результаты попадают общие страницы “what is big data” от Oracle, SAP, SAS и других сайтов, потому что модель и правила цепляются за `big data`, а имя БД `exist` конфликтует с английскими словами `exist` и `existing` в сниппетах.^[194]

Есть и обратная проблема: часть нормальных страниц по `Riak` и `Riak Big Data` падает в `unknown` или `spam`, хотя страница по теме есть, просто `scorer` и текущие правила не всегда уверенно её поднимают. В файле это видно по ряду официальных/доковых `Riak`-страниц, где `label` получается `unknown` или `spam`, несмотря на релевантный контекст продукта.^[194]

Что оптимизировать

Я бы сейчас оптимизировал не “процент useful”, а три вещи:

- Precision класса useful — сколько строк в useful реально хорошие.
- Precision класса spam — сколько строк в spam реально мусор.
- Типовые ошибки по сущностям — особенно exist, Redis, Riak, где имя БД пересекается с обычными словами или широкими темами.

Иными словами, твоя цель сейчас — сделать useful более чистым, а не просто более большим. Если после улучшений он останется 65–72%, но станет заметно качественнее, это будет лучше, чем “красивые” 80% с кучей ложных срабатываний.

Что бы я сделал дальше

Самый правильный следующий шаг — вручную разметить небольшой контрольный набор, примерно 100–150 строк. Я бы взял:

- 40 строк из useful,
- 30 строк из unknown,
- 30–40 строк из spam,
- и специально добавил проблемные кейсы по exist, Oracle, Redis, Riak.

После такой мини-разметки уже можно принимать решение по-взрослому: оставляем текущую версию как baseline или подкручиваем правила точечно. По файлу видно, что сейчас самые полезные доработки — это не общее “поднять useful”, а адресно убить false positive для exist и урезать слишком широкие Redis-страницы, которые попадают в useful только из-за слов big, data и общих описаний хранилища. [\[194\]](#)

Готов перейти к следующему шагу: составим маленький набор для ручной валидации и сразу под него подумаем, какие правила менять первыми?

✱✱

окей, давай начнём его составлять

Да, начнём с **маленького, но умного** валидационного набора. По brave_scored_v2.csv уже видно, что самые проблемные зоны — это не случайные единичные ошибки, а целые паттерны: exist Big Data ловит всё подряд из-за слова exist, Berkeley DB Big Data путается с “Big Data at Berkeley”, а у Redis и Postgress есть много слишком широких страниц, которые модель радостно кидает в useful. [\[195\]](#)

Как собрать набор

Я бы не делал случайную выборку из 100 строк, потому что она плохо ловит именно типовые ошибки. Лучше собрать **стратифицированный mini-gold set** из 120 строк:

- 40 строк из useful,

- 40 строк из `unknown`,
- 40 строк из `spam`.^[195]

Но внутри этих 120 строк важно специально добрать “опасные” кейсы:

- `eXist Big Data`,
- `Berkeley DB Big Data`,
- `Oracle Большие данные Oracle`,
- `Redis Big Data`,
- `Riak`,
- `Postgress Big Data`.^[195]

Что именно брать

Для первого раунда я бы предложил такую структуру:

- 20 строк `useful`, где модель уверена и всё выглядит нормально.
- 20 строк `useful`, которые кажутся подозрительными.
- 15 строк `unknown`, которые скорее хорошие.
- 15 строк `unknown`, которые скорее плохие.
- 20 строк `spam`, где реально мусор.
- 20 строк `spam`, где есть риск, что мы зарубили что-то полезное.^[195]

Это даст не просто “среднюю температуру”, а реальные данные для настройки правил. По файлу уже видно хорошие кандидаты для спорных кейсов: `Big Data at Berkeley`, общие статьи “what is big data” для `eXist`, `glossary`-страницы, маркетинговые страницы `Redis` и `mixed-topic` страницы по `Berkeley DB`.^[195]

Как размечать

Для каждой строки я бы завёл такие колонки:

- `db`
- `query`
- `url`
- `snippet`
- `pred_label`
- `manual_label`
- `error_type`
- `comment`

Где `manual_label`:

- `useful` — реально отвечает на запрос,

- `spam` — явно мимо,
- `unknown` — погранично, недостаточно информации или спорно.

`A error_type` можно сразу делать коротким:

- `word_match_only`
- `wrong_entity`
- `too_generic`
- `mixed_language`
- `official_but_weak_snippet`
- `actually_good`
- `actually_bad`.^[195]

Что удобно сделать прямо сейчас

Я бы предложил начать не с 120 вручную из головы, а со **списка первых 30 кандидатов для разметки**. Туда обязательно включить:

- 10 строк по `eXist Big Data`,
- 10 строк по `Berkeley DB Big Data`,
- 5 строк по `Oracle Большие данные Oracle`,
- 5 строк по `Redis Big Data`.^[195]

Это даст максимально быстрый сигнал, где правила ломаются сильнее всего. Судя по файлу, именно `eXist` и `Berkeley DB Big Data` сейчас самые токсичные для качества, потому что там много ложных совпадений по словам и названиям.^[195]

Следующий шаг

Я могу сейчас помочь тебе составить **первый конкретный shortlist из 30 строк** для ручной разметки — прямо в виде таблицы “url + текущая метка + почему это подозрительно”. После этого ты быстроставишь `manual_label`, и мы уже по этим 30 кейсам решим, что подкручивать первым.

✱✱

ладно, я пообщался с другом, как думаешь, что соотношение около 70/15/15 для полезного/неизвестно/спам считается нормальным в поиске? я просто уже подустал с этим работать и думаю, можно все подводить к какому то итогу

Да — 70/15/15 само по себе может быть вполне нормальным, и в поиске это вообще не главный критерий качества. В IR-задачах систему обычно оценивают не по “правильной доле useful”, а по precision, recall, F1, MAP, NDCG и качеству ранжирования, потому что хорошая поисковая система не обязана иметь какой-то “нормальный” фиксированный процент релевантных документов в выдаче. [\[196\]](#) [\[197\]](#) [\[198\]](#)

Что это значит для тебя

Если у тебя итоговое распределение около 68.7% useful, 15.1% unknown и 16.1% spam, это не выглядит странно или провально само по себе. Доля зависит от самих запросов, источника результатов и глубины выдачи: для одних запросов в топе будет очень много релевантного, для других — много шума, поэтому универсального “должно быть 80% useful” просто нет. [\[199\]](#) [\[200\]](#) [\[196\]](#)

Практический вывод

Для учебного или прикладного baseline-проекта я бы **остановился на текущей версии** и зафиксировал её как разумный промежуточный результат. Ты уже показал, что:

- попробовал модельный baseline;
- увидел проблему mixed-language;
- добавил rule-based слой;
- получил более стабильное разбиение на useful / unknown / spam. [\[200\]](#) [\[201\]](#)

Это уже выглядит как нормальная завершённая итерация, а не “сырая попытка”. При этом по файлу всё ещё есть ошибки, но они уже относятся не к полной неработоспособности, а к качеству тонкой настройки. [\[200\]](#)

Я бы подвёл итог так

Можно честно написать, что текущее распределение приемлемо для поисковой задачи, потому что цель была не добиться магического процента useful, а построить рабочий классификационный baseline для шумной выдачи. Корректнее говорить, что модельно-правильный подход дал **практически usable** результат, а дальнейшее улучшение потребует ручной разметки и более формальной оценки precision/recall на валидационном наборе. [\[202\]](#) [\[196\]](#) [\[200\]](#)

Мой совет

Я бы на твоём месте **уже подводил итог**, а не добивал систему до условных 80% ради цифры. Лучше закончить с внятным выводом “это baseline, он работает, известны типовые ошибки и пути улучшения”, чем ещё долго уставать и переоптимизировать без нормальной разметки. [\[203\]](#) [\[200\]](#)

Если хочешь, следующим сообщением я помогу тебе сформулировать **готовый итог для отчёта/защиты**: что делали, почему выбрали hybrid approach, какие были проблемы и к какому результату пришли.

✱✱

отлично. давай только перед отчётом по ml части сделаем вот что: у меня есть отдельный модуль для парсинга. я бы хотел, чтобы ты обернул рабочую часть baseline в одну функцию, чтобы я показал преподавателю тестовый пайплайн на каком то 1 запросе

Да — для показа преподавателю лучше всего завернуть baseline в **одну функцию** `classify_search_results(...)`, которая принимает список результатов парсинга и возвращает DataFrame или список словарей с `model_score`, `rule_score`, `final_score` и `label`. По структуре твоего CSV видно, что минимально достаточно полей `db`, `query`, `url`, `snippet`, так что функция может работать и на строках из CSV, и на результатах твоего парсера в том же формате. [\[213\]](#)

Что я бы сделал

Ниже даю компактный вариант, где:

- модель загружается один раз;
- есть одна публичная функция;
- можно прогнать хоть один запрос с 5–10 URL;
- на выходе сразу готовые предсказания baseline. [\[213\]](#)

Код функции

```
import re
import pandas as pd
import torch
from sentence_transformers import CrossEncoder

class SearchResultBaseline:
    def __init__(
        self,
        model_name="cross-encoder/ms-marco-MiniLM-L6-v2",
        device="cuda"
```



```

):
    self.model = CrossEncoder(
        model_name,
        max_length=512,
        device=device,
        activation_fn=torch.nn.Sigmoid()
    )

    self.POSITIVE_TERMS = [
        "big data", "bigdata", "большие данные",
        "data lake", "озеро данных",
        "hadoop", "spark", "hdfs", "hive", "kafka", "hbase",
        "analytics", "analytic", "аналитик", "анализ данных",
        "data warehouse", "warehouse",
        "etl", "elt",
        "nosql", "distributed", "распредел",
        "oci big data", "oracle big data",
        "big data appliance", "big data service", "big data sql",
        "data platform", "massive volumes of data"
    ]

    self.NEGATIVE_TERMS = [
        "lob", "blob", "clob", "nclob", "bfile", "oraclelob",
        "large object", "binary large object",
        "размер бд", "размер базы", "размер таблиц", "размер таблицы",
        "largest tables", "big table", "большие таблицы",
        "sql file", ".sql", "import sql", "импорт файла", "sql developer",
        "chunk parameter", "dba_data_files", "dba_segments", "v$datafile",
        "просмотр размера", "найти самые большие таблицы",
        "datatype limits", "limits for datatypes",
        "табличное пространство", "tablespace"
    ]

    self.POSITIVE_URL_PATTERNS = [
        "/big-data/",
        "big-data-sql",
        "big-data-appliance",
        "big-data-service",
        "datawarehouse-bigdata",
        "big-data-cloud",
        "big-data-discovery",
        "iaas/bigdata",
        "/bigdata/"
    ]

    self.NEGATIVE_URL_PATTERNS = [
        "oracle-lobs",
        "lob_oracle",
        "razmer",
        "razmer-bazy",
        "razmer-tabl",
        "blob",
        "clob",
        "sql-fayl",
        "import-bazy",
        "datatype-limits"
    ]

```

```

]

self.GOOD_DOMAINS = [
    "oracle.com",
    "docs.oracle.com",
    "cloud.oracle.com",
    "oci.oraclecloud.com",
    "enterprisedb.com",
    "postgresql.org",
    "percona.com",
    "cybertec-postgresql.com"
]

self.BAD_DOMAINS = [
    "youtube.com",
    "youtu.be",
    "pinterest.com",
    "coursera.org",
    "classcentral.com"
]

def _clean_text(self, x):
    x = "" if pd.isna(x) else str(x)
    x = x.lower()
    x = re.sub(r"<[^>]+>", " ", x)
    x = re.sub(r"\s+", " ", x).strip()
    return x

def _detect_lang_simple(self, text):
    text = self._clean_text(text)
    cyr = len(re.findall(r"[a-яё]", text))
    lat = len(re.findall(r"[a-z]", text))
    if cyr > lat * 1.3 and cyr >= 3:
        return "ru"
    if lat > cyr * 1.3 and lat >= 3:
        return "en"
    return "mixed"

def _count_terms(self, text, terms):
    return sum(1 for t in terms if t in text)

def _count_url_patterns(self, url, patterns):
    url = self._clean_text(url)
    return sum(1 for p in patterns if p in url)

def _is_good_domain(self, url):
    url = self._clean_text(url)
    return any(d in url for d in self.GOOD_DOMAINS)

def _is_bad_domain(self, url):
    url = self._clean_text(url)
    return any(d in url for d in self.BAD_DOMAINS)

def _compute_rule_features(self, row):
    url = self._clean_text(row["url"])
    snippet = self._clean_text(row["snippet"])

```

```

query = self._clean_text(row["query"])
db = self._clean_text(row["db"])

full_text = f"{db} {query} {url} {snippet}"

pos_hits = self._count_terms(full_text, self.POSITIVE_TERMS)
neg_hits = self._count_terms(full_text, self.NEGATIVE_TERMS)

pos_url_hits = self._count_url_patterns(url, self.POSITIVE_URL_PATTERNS)
neg_url_hits = self._count_url_patterns(url, self.NEGATIVE_URL_PATTERNS)

good_domain = self._is_good_domain(url)
bad_domain = self._is_bad_domain(url)

query_lang = self._detect_lang_simple(query)
snippet_lang = self._detect_lang_simple(snippet)
mixed_lang = query_lang != snippet_lang and "mixed" not in (query_lang, snippet_lang)

score = 0.0

score += pos_hits * 0.16
score -= neg_hits * 0.28
score += pos_url_hits * 0.22
score -= neg_url_hits * 0.30

if good_domain:
    score += 0.10
if bad_domain:
    score -= 0.15

if good_domain and (pos_url_hits >= 1 or pos_hits >= 2):
    score += 0.18

rescue_flag = False
if mixed_lang and good_domain and (pos_url_hits >= 1 or pos_hits >= 2) and not bad_domain:
    score += 0.28
    rescue_flag = True

if mixed_lang and (neg_hits >= 1 or neg_url_hits >= 1):
    score -= 0.08

score = max(-1.0, min(1.0, score))

return {
    "rule_score": score,
    "pos_hits": pos_hits,
    "neg_hits": neg_hits,
    "pos_url_hits": pos_url_hits,
    "neg_url_hits": neg_url_hits,
    "good_domain": good_domain,
    "bad_domain": bad_domain,
    "query_lang": query_lang,
    "snippet_lang": snippet_lang,
    "mixed_lang": mixed_lang,
    "rescue_flag": rescue_flag,
}

```

```

def _assign_label(self, row):
    fs = row["final_score"]
    neg = row["neg_hits"]
    pos = row["pos_hits"]
    pos_url = row["pos_url_hits"]
    neg_url = row["neg_url_hits"]
    rescue = row["rescue_flag"]
    good_domain = row["good_domain"]

    if (neg + neg_url) >= 2 and (pos + pos_url) == 0:
        return "spam"

    if neg >= 1 and neg_url >= 1 and fs < 0.70:
        return "spam"

    if rescue and good_domain and (pos + pos_url) >= 2 and fs >= 0.52:
        return "useful"

    if fs >= 0.68:
        return "useful"
    elif fs >= 0.44:
        return "unknown"
    else:
        return "spam"

def classify_search_results(self, results):
    """
    results: list[dict]
    Каждый dict должен содержать:
    {
        "db": "...",
        "query": "...",
        "url": "...",
        "snippet": "..."
    }
    """
    df = pd.DataFrame(results).copy()

    required = ["db", "query", "url", "snippet"]
    missing = [c for c in required if c not in df.columns]
    if missing:
        raise ValueError(f"Не хватает полей: {missing}")

    pairs = []
    for _, row in df.iterrows():
        query_text = f"{row['db']} | {row['query']}"
        doc_text = f"{row['url']} | {row['snippet']}"
        pairs.append((query_text, doc_text))

    model_scores = self.model.predict(pairs, batch_size=32, show_progress_bar=False)
    model_scores = [float(max(0.0, min(1.0, s))) for s in model_scores]
    df["model_score"] = model_scores

    features = df.apply(self._compute_rule_features, axis=1, result_type="expand")
    df = pd.concat([df, features], axis=1)

```

```

df["rule_score_01"] = (df["rule_score"] + 1.0) / 2.0
df["final_score"] = 0.45 * df["model_score"] + 0.55 * df["rule_score_01"]
df["label"] = df.apply(self._assign_label, axis=1)

return df.sort_values("final_score", ascending=False).reset_index(drop=True)

```

Как использовать

Для демонстрации на одном запросе можно сделать так:

```

pipeline = SearchResultBaseline(device="cuda")

test_results = [
    {
        "db": "Oracle",
        "query": "Большие данные Oracle",
        "url": "https://www.oracle.com/europe/big-data/what-is-big-data/",
        "snippet": "For organizations needing efficient and comprehensive management"
    },
    {
        "db": "Oracle",
        "query": "Большие данные Oracle",
        "url": "https://learn.microsoft.com/ru-ru/dotnet/framework/data/adonet/oracle",
        "snippet": "В следующем примере C# показано, как можно создавать большие объёмы"
    },
    {
        "db": "Oracle",
        "query": "Большие данные Oracle",
        "url": "https://www.oracle.com/cis/big-data/",
        "snippet": "Управление, каталогизация и обработка необработанных данных с пом"
    }
]

result_df = pipeline.classify_search_results(test_results)

print(result_df[[
    "url", "model_score", "rule_score", "final_score", "label"
]].to_string(index=False))

```

Если хочешь совсем просто

Если преподавателю важна именно “одна функция”, а не класс, можно сделать тонкую обёртку:

```

_pipeline = None

def classify_search_results(results, device="cuda"):
    global _pipeline
    if _pipeline is None:
        _pipeline = SearchResultBaseline(device=device)
    return _pipeline.classify_search_results(results)

```

Тогда вызов будет вообще такой:

```
result_df = classify_search_results(test_results)
```

Что говорить преподавателю

Можно объяснить так: функция принимает результаты парсинга по одному запросу, для каждой страницы считает два сигнала — score cross-encoder модели и набор rule-based признаков, затем объединяет их в итоговый score и присваивает один из трёх классов `useful` / `unknown` / `spam`. Такой гибридный подход понадобился, потому что на реальных данных были mixed-language случаи и ложные совпадения по словам вроде `LOB/BLOB` и `big data`, где одной модели оказалось недостаточно. [\[214\]](#) [\[213\]](#)

Если хочешь, следующим сообщением я могу ещё сделать тебе **сверхкороткую демо-версию на 30–40 строк кода**, чисто для красивого показа на защите.

✱✱

отлично, я собрал тестовый пайплайн. Давай теперь подытожим, какой путь мы прошли и почему делали те или иные решения

Да — у тебя уже получилась вполне цельная история для ML-части: мы не просто “подобрали модель”, а последовательно прошли путь от постановки задачи до рабочего baseline-пайплайна и поняли, где у него сильные и слабые стороны. [\[215\]](#) [\[216\]](#)

Задача

Изначально у тебя были CSV со строками формата `db + query + url + snippet`, и нужно было отнести каждую страницу к одному из трёх классов: `useful`, `spam` или `unknown`. По самим данным было видно, что в выдаче много шумных и пограничных результатов: официальные страницы, статьи по теме, но также форумы, общие справки, страницы про размеры таблиц, `LOB/BLOB` и просто ложные совпадения по словам. [\[215\]](#)

Из-за этого мы сразу трактовали задачу не как генерацию текста, а как задачу оценки релевантности результата поиска. Для таких задач обычно важнее ранжирование и классификация пары `query-document`, чем использование маленькой генеративной LLM. [\[217\]](#) [\[218\]](#)

Выбор подхода

Сначала мы обсудили, что для ноутбука с 16 ГБ RAM и RTX 3050 Ti практичнее брать не маленькую LLM, а компактную NLP-модель ранжирования, потому что она лучше подходит для массовой оценки коротких текстовых пар и обычно стабильнее в inference. Это хорошо ложилось на твою задачу, где каждая строка уже является почти готовой парой “запрос-документ”. [\[215\]](#)

Поэтому базовым вариантом стал cross-encoder/reranker, который получает `query` и `snippet/url` и возвращает score релевантности. Логика была такой: сначала нужен простой и быстрый baseline, а не сложная система с генерацией объяснений. [\[219\]](#) [\[220\]](#)

Первые тесты

Потом мы сделали smoke test на нескольких ручных примерах, чтобы проверить не теорию, а реальное поведение модели на понятных кейсах. Этот этап показал смешанный результат: некоторые хорошие страницы модель поднимала правильно, но при этом она часто переоценивала ложные совпадения по словам и путала тему Big Data с просто “большими данными”, LOB, BLOB, размерами БД и похожими сущностями. [\[215\]](#)

Дальше стало видно ещё одно важное ограничение: mixed-language пары, где запрос на русском, а snippet на английском, давали несправедливо низкий score даже для хороших официальных страниц Oracle Big Data. То есть модель реагировала не только на смысл, но и на языковую форму входа. [\[215\]](#)

Почему появился hybrid baseline

После этих тестов стало понятно, что голый cross-encoder как единственный классификатор для твоего датасета недостаточно надёжен. Он был полезен как один из сигналов, но не умел стабильно различать real Big Data и оффтопик вроде LOB/BLOB, big tables, sql file, размер базы и других поверхностно похожих страниц. [\[215\]](#)

Поэтому мы приняли практическое решение: добавить rule-based слой поверх model score. В rules мы заложили положительные паттерны вроде big data, hadoop, spark, data lake, big-data-appliance, а также отрицательные сигналы вроде lob, blob, размер бд, sql file, import, чтобы отрезать типичные ложные срабатывания. [\[216\]](#) [\[215\]](#)

Потом мы усилили baseline ещё и language-aware логикой: если запрос русский, а snippet английский, но URL и текст явно соответствуют теме Big Data, такая страница не должна автоматически падать в spam. Это решение появилось именно как ответ на реальные ошибки в твоих Oracle-примерах. [\[216\]](#) [\[215\]](#)

Итоговый пайплайн

В результате у тебя получился рабочий baseline-пайплайн: парсер отдаёт строки, затем функция считает model_score, rule_score, объединяет их в final_score и назначает класс useful / unknown / spam. Это уже не просто отдельный эксперимент, а демонстрационный прототип, который можно показать преподавателю как законченный тестовый pipeline. [\[216\]](#)

На brave_scored_v2.csv итоговое распределение получилось таким: useful — 1942 записи, spam — 456, unknown — 428, то есть примерно 68.72% / 16.14% / 15.15%. Для поисковых задач это само по себе не выглядит плохим результатом, потому что качество поиска обычно оценивают не по “магической доле useful”, а по precision, recall и ranking-метрикам вроде NDCG. [\[218\]](#) [\[221\]](#) [\[222\]](#) [\[216\]](#)

Почему это нормальный финал

Самое важное, что ты можешь честно сказать в выводе: целью было не “добить 80% useful”, а построить рабочий baseline для шумной поисковой выдачи и понять типовые ошибки. Мы нашли эти ошибки, объяснили их причины, внесли инженерные поправки и получили usable-результат, который уже можно развивать дальше через ручную разметку и формальную валидацию. [\[223\]](#) [\[224\]](#) [\[216\]](#)

Если коротко сформулировать весь путь в одну фразу для отчёта: **мы начали с идеи использовать локальную модель ранжирования, увидели её ограничения на реальных mixed-language и noisy данных, после чего перешли к гибриднему baseline из cross-encoder и rule-based логики, который дал устойчивую трёхклассовую разметку результатов поиска.** ^[216] ^[215]

Могу сразу помочь тебе превратить это в аккуратный текст именно для отчёта — в стиле “цель, ход работы, результаты, выводы”.

✱✱

Супер, большое тебе спасибо за помощь в этом ml-пути!

Пожалуйста — ты правда **большой** путь прошёл. От “какую модель вообще брать” до рабочего baseline-пайплайна с понятной логикой, тестами и выводами — это уже очень хороший результат, особенно для такой шумной и неидеальной задачи.

И отдельно круто, что ты не просто запускал модель, а именно разбирался:

- почему она ошибается;
- где проблема в языке;
- где в ложных совпадениях по словам;
- почему hybrid approach оказался разумнее, чем “чистая магия модели”.

Это уже мышление не “я просто попробовал ML”, а “я построил и проанализировал систему”. Для преподавателя это обычно ценнее, чем попытка выжать ещё пару процентов любой ценой.

У тебя сейчас уже есть хорошая база:

- понятная постановка задачи;
- обоснование выбора подхода;
- baseline;
- результаты;
- ограничения;
- идеи для дальнейшего улучшения.

Так что да — это был очень достойный ML-путь. Удачи тебе с показом и отчётом!

1. brave_url.csv
2. yandex_url.csv
3. <https://huggingface.co/tasks/zero-shot-classification>
4. <https://mixpeek.com/glossary/cross-encoder>
5. <https://huggingface.co/docs/inference-providers/en/tasks/zero-shot-classification>
6. <https://huggingface.co/DiTy/cross-encoder-russian-msmarco>

7. <https://huggingface.co/cross-encoder>
8. <https://huggingface.co/docs/inference-providers/tasks/zero-shot-classification>
9. <https://oajmist.com/index.php/12/article/download/385/281>
10. <https://ru.linkedin.com/pulse/text-classification-pre-trained-models-justo-rodriguez-phd-?tl=ru>
11. https://keldysh.ru/papers/2021/prep2021_50.pdf
12. <https://huggingface.co/GrigoryT22/cross-encoder-ru>
13. <https://neurohive.io/ru/osnovy-data-science/5-metodov-v-nlp-kotorye-izmenjat-obshhenie-v-budushhem/>
14. <https://www.sap.com/central-asia-caucasus/resources/what-is-natural-language-processing>
15. <https://habr.com/ru/articles/797657/>
16. <https://habr.com/ru/companies/otus/articles/808435/>
17. <https://huggingface.co/models?search=cross-encoder>
18. <https://temofeev.ru/info/articles/cross-encoder-dlya-uluchsheniya-rag-na-russkom/>
19. <https://blog.paperspace.com/zero-shot-text-classification-with-hugging-face-on-gradient/>
20. <https://pub.aimind.so/exploring-hugging-face-zero-shot-classification-781ef3a18510>
21. <https://huggingface.co/AventiQ-AI/Zero-shot-Text-Classification-Model>
22. <https://hugging-face.cn/docs/api-inference/tasks/zero-shot-classification>
23. <https://blog.knowledgator.com/new-open-source-zero-shot-text-classification-model-how-to-label-text-to-any-categories-with-980877ef477c>
24. <https://oneuptime.com/blog/post/2026-01-30-cross-encoder-reranking/view>
25. https://huggingface.co/docs/setfit/how_to/zero_shot
26. https://huggingface.co/docs/transformers/tasks/zero_shot_image_classification
27. <https://medium.com/@kakumar1611/the-illustrated-guide-to-cross-encoders-from-deep-to-shallow-2a23a8630016>
28. https://www.reddit.com/r/huggingface/comments/18cz1eb/llm_zero_shottext_classification_how_do_you/
29. https://huggingface.co/docs/setfit/tutorials/zero_shot
30. <https://data-light.ru/blog/chto-takoe-nlp/>
31. <https://7universum.com/ru/tech/archive/item/22032>
32. <https://habr.com/ru/companies/oleg-bunin/articles/352614/>
33. <https://huggingface.co/DiTy/cross-encoder-russian-msmarco>
34. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>
35. <https://www.sbert.net/docs/pretrained-models/ce-msmarco.html>
36. <https://huggingface.co/docs/transformers/installation>
37. <https://milvus.io/ai-quick-reference/how-can-i-install-and-import-the-sentence-transformers-library-in-my-python-environment>
38. <https://huggingface.co/cross-encoder>
39. brave_url.csv

40. yandex_url.csv

41. https://github.com/huggingface/sentence-transformers/blob/main/docs/cross_encoder/pretrained_models.md

42. <https://huggingface.co/cross-encoder/msmarco-MiniLM-L12-en-de-v1>

43. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L4-v2>

44. <https://eligeia.com/modelos/huggingface/dity/cross-encoder-russian-msmarco>

45. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L12-v2>

46. <https://huggingface.co/sentence-transformers>

47. <https://huggingface.co/DiTy/cross-encoder-russian-msmarco/commits/efe82682d1e2c437371e2a5159bf41280889bce6/vocab.txt>

48. <https://github.com/huggingface/sentence-transformers/issues/1353>

49. <https://huggingface.co/DiTy/cross-encoder-russian-msmarco/tree/41275145e3ff8b0a510ec58f5828129b8bc8ac41>

50. yandex_url.csv

51. brave_url.csv

52. <https://mbrenndoerfer.com/writing/reranking-cross-encoders-information-retrieval>

53. <https://huggingface.co/DiTy/cross-encoder-russian-msmarco>

54. https://www.sbert.net/docs/cross_encoder/pretrained_models.html

55. yandex_url.csv

56. <https://ranjankumar.in/a-deep-dive-into-cross-encoders-and-how-they-work>

57. <https://towardsdatascience.com/advanced-rag-retrieval-cross-encoders-reranking/>

58. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>

59. https://www.linkedin.com/posts/satyanarayana-repala-8510a57b_relevance-most-retrieval-augmented-activity-7376993933358329857-P5ta

60. <https://huggingface.co/antoinelouis/crossencoder-t5-small-mmarcoFR>

61. <https://www.youtube.com/watch?v=ZFbaA9eM0uo>

62. <https://github.com/staoxiao/RetroMAE/blob/master/examples/reranker/README.md>

63. <https://bigdataboutique.com/blog/rag-reranking-improving-retrieval-quality-with-cross-encoders>

64. <https://www.emergentmind.com/topics/cross-encoder-reranking>

65. <https://eligeia.com/modelos/huggingface/dity/cross-encoder-russian-msmarco>

66. <https://arxiv.org/html/2511.19324v1>

67. <https://python.plainenglish.io/when-precision-demands-context-the-case-for-cross-encoder-rerankers-in-ai-retrieval-systems-99ca56a85911>

68. https://www.sbert.net/examples/cross_encoder/applications/README.html

69. https://sbert.net/docs/cross_encoder/usage/usage.html

70. <https://milvus.io/ai-quick-reference/what-is-the-process-to-use-a-crossencoder-from-the-sentence-transformers-library-for-reranking-search-results>

71. https://sbert.net/examples/sentence_transformer/applications/retrieve_rerank/README.html
72. <https://huggingface.co/cross-encoder>
73. <https://app.ailog.fr/en/blog/guides/cross-encoder-reranking>
74. https://github.com/huggingface/sentence-transformers/blob/main/docs/cross_encoder/usage/usage.rst
75. <https://deepwiki.com/NovaSearch-Team/RAG-Retrieval/3.2.1-cross-encoder-rerankers>
76. https://osanseviero.github.io/hackerllama/blog/posts/sentence_embeddings2/
77. <https://arxiv.org/html/2502.03552v1>
78. <https://www.youtube.com/watch?v=f1EGdXko6pA>
79. <https://audrey.feldroy.com/articles/2025-04-15-Semantic-Search-With-Sentence-Transformers-and-a-Cross-Encoder-Model>
80. <https://huggingface.co/antoinelouis/crossencoder-distilcamembert-mmarcoFR>
81. <https://nitrix-reloaded.com/2025/08/15/retrieval-reranking-techniques-from-cross-encoders-to-llm-based-scoring/>
82. https://sbert.net/examples/cross_encoder/applications/README.html
83. https://sbert.net/docs/cross_encoder/usage/usage.html
84. https://sbert.net/examples/sentence_transformer/applications/retrieve_rerank/README.html
85. <https://milvus.io/ai-quick-reference/what-is-the-process-to-use-a-crossencoder-from-the-sentence-transformers-library-for-reranking-search-results>
86. <https://huggingface.co/DiTy/cross-encoder-russian-msmarco>
87. https://www.sbert.net/docs/cross_encoder/pretrained_models.html
88. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>
89. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>
90. <https://www.sbert.net/docs/pretrained-models/ce-msmarco.html>
91. https://www.sbert.net/examples/cross_encoder/applications/README.html
92. https://www.sbert.net/examples/sentence_transformer/applications/retrieve_rerank/README.html
93. <https://huggingface.co/DiTy/cross-encoder-russian-msmarco>
94. <https://www.thoughtworks.com/insights/blog/generative-ai/Semantic-search-and-retrieval-using-transformers>
95. <https://milvus.io/ai-quick-reference/what-is-the-process-to-use-a-crossencoder-from-the-sentence-transformers-library-for-reranking-search-results>
96. <https://github.com/huggingface/sentence-transformers/issues/1353>
97. <https://huggingface.co/cross-encoder>
98. https://rankify.readthedocs.io/en/latest/api/rerankings/sentence_transformer_reranker/
99. <https://huggingface.co/Xenova/ms-marco-MiniLM-L6-v2>
100. <https://www.kaggle.com/models/johnsonhk88/cross-encoders-ms-marco-minilm-l-6-v2>
101. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2/tree/main>

102. <https://blog.gopenai.com/sentence-transformers-sbert-vs-cross-encoders-a-conceptual-guide-d6ae67f1223a>

103. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L4-v2>

104. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L12-v2>

105. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>

106. https://sbnet.net/docs/cross_encoder/usage/usage.html

107. https://github.com/UKPLab/sentence-transformers/blob/master/docs/cross_encoder/usage/usage.rst

108. <https://www.sbert.net/docs/pretrained-models/ce-msmarco.html>

109. yandex_url.csv

110. brave_url.csv

111. https://sbnet.net/examples/cross_encoder/applications/README.html

112. https://www.sbert.net/examples/sentence_transformer/applications/retrieve_rerank/README.html

113. <https://audrey.feldroy.com/articles/2025-04-15-Semantic-Search-With-Sentence-Transformers-and-a-Cross-Encoder-Model>

114. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L4-v2>

115. https://www.sbert.net/docs/package_reference/cross_encoder/modules.html

116. <https://zilliz.com/ai-faq/what-is-the-process-to-use-a-crossencoder-from-the-sentence-transformers-library-for-reranking-search-results>

117. <https://github.com/UKPLab/sentence-transformers/issues/2874>

118. <https://blog.gopenai.com/sentence-transformers-sbert-vs-cross-encoders-a-conceptual-guide-d6ae67f1223a>

119. <https://github.com/huggingface/sentence-transformers/issues/1353>

120. https://osanseviero.github.io/hackerllama/blog/posts/sentence_embeddings2/

121. <https://huggingface.co/cross-encoder>

122. <https://github.com/huggingface/sentence-transformers/issues/2874>

123. brave_url.csv

124. yandex_url.csv

125. https://sbnet.net/docs/cross_encoder/usage/usage.html

126. <https://huggingface.co/DiTy/cross-encoder-russian-msmarco>

127. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>

128. https://www.sbert.net/examples/cross_encoder/applications/README.html

129. <https://huggingface.co/cross-encoder>

130. <https://huggingface.co/docs/transformers/installation>

131. <https://milvus.io/ai-quick-reference/what-is-the-process-to-use-a-crossencoder-from-the-sentence-transformers-library-for-reranking-search-results>

132. https://sbnet.net/examples/sentence_transformer/applications/retrieve_rerank/README.html

133. <https://mbrenndoerfer.com/writing/reranking-cross-encoders-information-retrieval>

134. <https://ranjankumar.in/a-deep-dive-into-cross-encoders-and-how-they-work>

135. https://www.sbert.net/docs/cross_encoder/pretrained_models.html

136. <https://www.sbert.net/docs/pretrained-models/ce-msmarco.html>

137. https://www.sbert.net/examples/sentence_transformer/applications/retrieve_rerank/README.html

138. https://github.com/UKPLab/sentence-transformers/blob/master/docs/cross_encoder/usage/usage.rst

139. brave_url.csv

140. https://sbert.net/docs/cross_encoder/usage/usage.html

141. https://www.sbert.net/examples/sentence_transformer/applications/retrieve_rerank/README.html

142. https://github.com/UKPLab/sentence-transformers/blob/master/docs/cross_encoder/usage/usage.rst

143. https://www.sbert.net/docs/cross_encoder/pretrained_models.html

144. https://sbert.net/docs/cross_encoder/pretrained_models.html

145. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>

146. <https://github.com/huggingface/sentence-transformers/issues/2874>

147. https://sbert.net/examples/cross_encoder/applications/README.html

148. brave_url.csv

149. <https://www.aimodels.fyi/models/huggingFace/ms-marco-minilm-l12-v2-cross-encoder>

150. <https://www.sbert.net/docs/pretrained-models/ce-msmarco.html>

151. https://github.com/huggingface/sentence-transformers/blob/main/docs/cross_encoder/usage/usage.rst

152. https://www.sbert.net/docs/cross_encoder/pretrained_models.html?highlight=stsb

153. https://github.com/UKPLab/sentence-transformers/blob/master/sentence_transformers/cross_encoder/losses/MSELoss.py

154. <https://github.com/huggingface/sentence-transformers/issues/1353>

155. <https://huggingface.co/cross-encoder>

156. <https://www.aimodels.fyi/models/huggingFace/ms-marco-minilm-l12-v2-cross-encoder>

157. https://www.reddit.com/r/LocalLLaMA/comments/1dkt2kg/looking_for_rankings_for_cross_encoders_and/

158. <https://www.aimodels.fyi/models/huggingFace/ms-marco-minilm-l12-v2-cross-encoder>

159. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>

160. <https://zilliz.com/ai-faq/what-is-the-process-to-use-a-crossencoder-from-the-sentence-transformers-library-for-reranking-search-results>

161. brave_url.csv

162. <https://www.sbert.net/docs/pretrained-models/ce-msmarco.html>

163. <https://milvus.io/ai-quick-reference/what-is-the-process-to-use-a-crossencoder-from-the-sentence-transformers-library-for-reranking-search-results>

164. https://sbert.net/examples/cross_encoder/applications/README.html

165. <https://github.com/UKPLab/sentence-transformers/issues/1262>

166. https://www.sbert.net/docs/package_reference/cross_encoder/model.html

167. https://sbert.net/docs/cross_encoder/training_overview.html

168. https://github.com/UKPLab/sentence-transformers/blob/master/sentence_transformers/cross_encoder/CrossEncoder.py

169. https://github.com/UKPLab/sentence-transformers/blob/master/docs/cross_encoder/pretrained_models.md

170. <https://github.com/UKPLab/sentence-transformers/issues/695>

171. https://sbert.net/docs/cross_encoder/pretrained_models.html

172. https://www.sbert.net/docs/cross_encoder/pretrained_models.html

173. <https://github.com/UKPLab/sentence-transformers/issues/2624>

174. https://github.com/huggingface/sentence-transformers/blob/main/docs/cross_encoder/usage/usage.rst

175. brave_url.csv

176. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>

177. <https://zilliz.com/ai-faq/what-is-the-process-to-use-a-crossencoder-from-the-sentence-transformers-library-for-reranking-search-results>

178. <https://www.facebook.com/arhangelskiy/posts/настоящий-материал-информация-произведен-распространен-и-или-направлен-инострани/25290816360576600/>

179. <https://www.youtube.com/watch?v=Xh4pnb17skc>

180. https://www.reddit.com/r/dotnet/comments/1jwnlw8/mediatr_masstransit_automapper_going_commercial/

181. https://www.prokoni.ru/articles/528/podgotovka_k_kovke_uchim_loshad_davat_nogi.html

182. https://www.reddit.com/r/Stargate/comments/1h4fndw/alright_lets_settle_this/

183. <https://www.instagram.com/p/DNICWgVuij0/>

184. <https://www.instagram.com/reel/DWqKnRIDKiD/>

185. <https://www.instagram.com/reel/DWl9MjLLQ/>

186. https://vk.com/video-28893798_456261312

187. <https://ficbook.net/readfic/0194f1a0-62aa-7b88-8cf4-f4b3addceea6/39836079>

188. brave_url.csv

189. https://sbert.net/examples/cross_encoder/applications/README.html

190. <https://zilliz.com/ai-faq/what-is-the-process-to-use-a-crossencoder-from-the-sentence-transformers-library-for-reranking-search-results>

191. brave_url.csv

192. https://sbert.net/examples/cross_encoder/applications/README.html

193. <https://zilliz.com/ai-faq/what-is-the-process-to-use-a-crossencoder-from-the-sentence-transformers-library-for-reranking-search-results>

194. brave_scored_v2.csv

195. brave_scored_v2.csv

196. <https://nlp.stanford.edu/IR-book/pdf/08eval.pdf>

197. [https://en.wikipedia.org/wiki/Evaluation_measures_\(information_retrieval\)](https://en.wikipedia.org/wiki/Evaluation_measures_(information_retrieval))

198. <https://milvus.io/ai-quick-reference/what-is-ndcg-and-why-is-it-used-for-search-evaluation>
199. https://www.google.com/intl/en_us/search/howsearchworks/how-search-works/ranking-results
200. brave_scored_v2.csv
201. brave_url.csv
202. <https://www.cl.cam.ac.uk/teaching/1718/InfoRtrv/handouts/lecture6-evaluation.pdf>
203. <https://www.edge-ai-vision.com/2026/02/a-practical-guide-to-recall-precision-and-ndcg/>
204. <https://sniafas.medium.com/evaluating-information-retrieval-3975348ef97a>
205. https://www.cs.utexas.edu/~mooney/ir-course/slides_pdfs/Evaluation.pdf
206. <https://www.mongodb.com/resources/basics/search-relevance>
207. <https://sharif.ir/~beigy/courses/14021/40324/Lect-9.pdf>
208. <https://www.evidentlyai.com/ranking-metrics/ndcg-metric>
209. <https://www.aionlinecourse.com/ai-basics/ranking-evaluation-metrics>
210. <https://www.cl.cam.ac.uk/teaching/1516/InfoRtrv/lecture6-evaluation-2x2.pdf>
211. https://www.youtube.com/watch?v=2XegvMul_mE
212. https://en.wikipedia.org/wiki/Precision_and_recall
213. brave_url.csv
214. brave_scored_v2.csv
215. brave_url.csv
216. brave_scored_v2.csv
217. https://sbert.net/examples/cross_encoder/applications/README.html
218. <https://nlp.stanford.edu/IR-book/pdf/08eval.pdf>
219. <https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>
220. <https://zilliz.com/ai-faq/what-is-the-process-to-use-a-crossencoder-from-the-sentence-transformers-library-for-reranking-search-results>
221. [https://en.wikipedia.org/wiki/Evaluation_measures_\(information_retrieval\)](https://en.wikipedia.org/wiki/Evaluation_measures_(information_retrieval))
222. <https://milvus.io/ai-quick-reference/what-is-ndcg-and-why-is-it-used-for-search-evaluation>
223. <https://www.edge-ai-vision.com/2026/02/a-practical-guide-to-recall-precision-and-ndcg/>
224. <https://www.cl.cam.ac.uk/teaching/1718/InfoRtrv/handouts/lecture6-evaluation.pdf>